

Session 1 speaking script: The Evolution of C++, Session 1: The Everyday Language

How to use this script

- **Bold lines** are the must-say sentences. If you say nothing else on a slide, say those.
- Plain paragraphs are the talk track, written to be read aloud as is. Paraphrase freely once rehearsed.
- (pause) and (beat) are deliberate stops. A pause is two full seconds of silence; a beat is one. Over Teams, silence feels longer to you than to them. Take it anyway.
- >> DO: lines are actions: terminal commands, editor moves, Compiler Explorer edits, with the output to expect and a recovery line.
- >> ASK: lines are questions to the room, with the answer you expect and what to say next. Over Teams, ask for answers in chat and read two or three aloud.
- >> IF AHEAD: is 60 to 120 seconds of real extra depth. >> IF BEHIND: is the one sentence to say instead of the slide's talk track.

Pace plan for a fast speaker

Checkpoint slide	Target clock
1. Title	0:00
5. The toolchain (start the two-minute check)	0:05
7. What this course assumes you know (calibration starts)	0:10
15. Calibration takeaway	0:28
16. C++14 in one slide	0:30
26. C++14 takeaway (deck time check)	0:53
27. C++17 in one slide	0:55
34. [[nodiscard]]	1:07
41. C++17 takeaway (deck time check)	1:20
43. Three-way comparison, 1	1:22
53. C++20 and C++23 takeaway	1:35
54. Exercise launch	1:35
55. Session 1 takeaway	1:55
End	2:00

If you hit a checkpoint more than 3 minutes early, use the IF AHEAD material in the next segment rather than speeding on.

Slow down deliberately on these four:

- **Slide 9, when `std::move` does nothing.** It is the first slide where "I know C++11" meets "I did not know that." The const case is silent, and moved-from state is the foundation for Session 2. If hands stayed down for move semantics on slide 7, this is where you spend the slack.
- **Slide 36, guaranteed copy elision.** The idea that a prvalue is a recipe, not an object, is genuinely new to most people, and it is the reason the `return std::move` warning on slide 9 exists.
- **Slides 44 and 45, operator<=> rewriting and categories.** Two rules people get wrong for years: `==` is separate from `<=>`, and member declaration order is the comparison order. Both show up in exercise task 2 twenty minutes later.

Before class checklist

- Build and test once, from the repo root, at least an hour before: `cmake -S . -B build -G Ninja && cmake --build build && ctest --test-dir build --output-on-failure`. Confirm `build/exercises/s01-modernize-syntax/s01_starter` exists and runs on the sample data (see the path note in Deck issues; run it once from the root and once from the exercise folder so you know which path works).
- Terminal > Run Task > slides: `serve` (live reload), then Simple Browser at `http://localhost:8080/01-everyday-language.md`, dragged to the left group. Terminal at the bottom, cwd at the repo root.
- Editor tabs on the right, in this order:
`exercises/s01-modernize-syntax/starter/record.h`,
`demos/s01/auto_pitfalls.cpp`, `move_does_nothing.cpp`,
`escaping_lambda.cpp`, `generic_lambda.cpp`, `structured_bindings.cpp`,
`nodiscard.cpp`, `copy_elision.cpp`, `ctad.cpp`, `from_chars.cpp`,
`designated_init.cpp`, `spaceship.cpp`, `spaceship_details.cpp`,
`range_for_init.cpp`, `small_cpp23.cpp`. Close everything else.
- The demo files still say Compiler Explorer: <add short link>, so build your own tabs. Preload in the godbolt Simple Browser tab, x86-64 GCC 14, flags `-std=c++23 -Wall -Wextra -Wpedantic -Werror`, "Execute the code" output pane on:
 1. `escaping_lambda.cpp` with `make_logger_bad("boom")()`; uncommented and `-fsanitize=address -g` added **in the Executor pane's own "Compiler options" box**. The course links open two panes, and each has its own flags: the top (assembly) pane's flags do not reach the Executor, so flags typed there compile the asm with ASan and run the program without it, which prints nothing. The report goes to stderr, in red, and [ok] from the good logger never appears because ASan aborts before stdout is flushed.
 2. `copy_elision.cpp` as is (you will flip `-std=c++23` to `-std=c++14` live).
 3. `designated_init.cpp` (you will uncomment the wrong-order line live).
 4. `spaceship_details.cpp` with this added to main: `Bad x{}, y{}; x.value = 2.0; x.ts = 1; y.value = 1.0; y.ts = 2; std::printf("x<y %d\n", x < y);` Prints `x<y 0`; after swapping the two members of `Bad` it prints `x<y 1`.

5. `range_for_init.cpp` with the bug line uncommented and given a body (`{ std::printf("%s\n", n.c_str()); }`), plus `-fsanitize=address -g`. Take `-Werror` off this tab, because GCC's `-Wdangling-reference` (in `-Wall`) flags the line before ASan gets a chance.
 - Know your expected outputs (verified with GCC and Clang):
`demo_s01_auto_pitfalls` prints `cfg cfg! cfg! 42 2 3 2`;
`demo_s01_move_does_nothing` prints `100, 100, 100, 0`;
`demo_s01_spaceship_details` prints `1 1 1`; `demo_s01_ctad` prints `1 2.5 3 3 2`.
 - Do not run `demo_s01_from_chars` live: its printed values differ between GCC and Clang (see Deck issues).
 - Exercise: confirm the starter is still built as C++11 in its `CMakeLists.txt`, and decide now what you will tell people about switching it to C++23 before task 2. Have `solution/record.h` ready in a tab you can open at 1:55.
 - Post the repo link and `exercises/s01-modernize-syntax/README.md` path in the Teams chat before you start.
-

The script

1. The Evolution of C++ (title) • target 0:00, ~2 min

Good morning, everyone. Thanks for giving this two hours.

I'm Trevor Bakker. Day job, I'm a software and cyber ETM here. On the side I teach Operating Systems and Information Security as an adjunct at UT Arlington. So I spend a lot of time explaining systems to people who are going to go build them, and that's the spirit of this course.

This course is about one question: what changed in C++ after C++11, and which of it should you be using on Monday?

Quick poll before we start. In the chat, type the last C++ standard you actually shipped production code with. Just the number. 98, 03, 11, 14, 17, 20.

(pause)

DO: Watch the chat for 15 seconds. Read three answers aloud, including the oldest one.

That's about what I expected. Most of us live somewhere between 11 and 17, and a few of us are still on 98 because a certified toolchain or a customer said so. That's fine. That's the audience this was built for.

Here's the frame for all five sessions. We're covering C++14, 17, 20, and 23. But we're **not** doing one standard per session. We're going by theme. Today is the everyday language, the small syntax that touches nearly every function you write. Session 2 is vocabulary types: `optional`, `variant`, `expected`, `string_view`, `span`, `format`. Session 3 is compile-time and generic programming: `constexpr`, concepts. Session 4 is ranges. Session 5 is concurrency, coroutines, modules, and how to actually adopt all of this.

Every feature slide carries a badge saying which standard it came from, so you never lose the chronology. And the feature timeline handout is the map. Keep it open.

(beat)

Let's look at the next two hours.

2. Agenda • target 0:02, ~1 min

Six blocks. Ten minutes of setup, which you're in now. Twenty minutes of C++11 calibration, where we make sure we agree on the baseline. Twenty-five on C++14. Twenty-five on C++17. Fifteen on the small C++20 and 23 features. Then twenty minutes of hands-on exercise.

The exercise is cumulative: the program you modernize today is the program you carry through all five sessions.

So please open `exercises/s01-modernize-syntax/README.md` now. The path's in the chat. You don't have to read it yet. Just get it open, and if you haven't built the repo, start that build in the background while I talk. We don't have a scheduled break, so the time to build is now, not at 1:35.

(pause)

Why themes instead of standards? That's the next slide.

IF BEHIND: "Six blocks, exercise at the end, open the README now and start your build."

3. Why not one session per standard? • target 0:03, ~1.5 min

Look at the right-hand column. This is a rough share of the "must know" material in each standard.

C++14 is about 5 percent. It was a polish release. C++17 is about a quarter, and it's the stuff you'll type every day. C++20 is half. It's the biggest change to the language since C++11: concepts, ranges, coroutines, modules, all in one release. C++23 is the rest, and a lot of it is finishing what 20 started.

If I gave each standard a session, C++14 would get two hours it doesn't need, and C++20 would get two hours it can't fit in.

But the bigger reason is the line under the table. **The important stories cross standards.** `constexpr` changed in every one of these releases. Relaxed in 14. `if constexpr` and `constexpr lambdas` in 17. Allocation and virtual calls in 20. Most of the remaining restrictions gone in 23. Lambdas: generic in 14, `constexpr` in 17, template lambdas in 20. Ranges: arrived in 20, became comfortable in 23.

If I taught by year, I'd teach `constexpr` four times, and you'd never see the whole arc in one place.

(pause)

So: themes, with badges. Here's how to read one of those badges.

IF BEHIND: "C++20 alone is half the material, and the big stories like constexpr span every standard, so we go by theme."

4. How to read a feature slide • C++17 (sample badge) • target 0:04, ~1 min

Every feature slide has three parts.

The badge, top right. That's the standard that introduced the feature. **The badge is also your answer to "can I use this under our flags?"** If your project builds with `-std=c++17`, anything badged 20 or 23 is off the table until that flag moves.

The problem line, in italics under the title. That's what was wrong before. If the problem line doesn't describe a problem you have, the feature can wait.

(pause)

And the code. Most code blocks are excerpts from files in `demos/s01`, pulled in by a script, so they compile cleanly under warnings-as-errors on both GCC and Clang. A handful are hand-typed because they show C++11-only forms or fragments, and the speaker notes say which. When you see `. . .` in a block, that's a fragment, not code.

Before and after slides put the C++11 way on the left, the modern way on the right.

(beat)

Speaking of compiling, let's make sure yours does.

IF BEHIND: "Badge is the standard, italic line is the problem, code comes from compiled demo files."

5. The toolchain • target 0:05, ~3 min (includes a 2-minute wait)

Everything in this course is `-std=c++23`, on GCC 14 or Clang 18. Either works. If you can, have both, because they disagree occasionally and the disagreement is instructive.

Warnings are on and they are errors: `-Wall -Wextra -Wpedantic -Werror`. Treat the compiler as a participant in this course. Several slides today show a mistake by showing the compiler refusing it. That's the same posture a lot of our programs take in CI, so it should feel familiar.

If you can't build locally, Compiler Explorer, godbolt.org, will do everything you need today. And the repo has a Dockerfile that pins the exact toolchain if your workstation is behind.

One caution. Some C++23 *library* pieces are uneven between `libstdc++` and `libc++`. The toolchain support matrix handout lists them. Nothing we use today is missing on either, with one footnote on `from_chars` that I'll point out when we get there.

So, the two-minute check. Run the command on the slide from the repo root: `configure, build, test`.

DO: Paste into chat: `cmake -S . -B build && cmake --build build && ctest --test-dir build`. Then actually wait. Start a visible timer if it helps you.

While that runs: if you get a configure error about the CMake version, you need 3.28 or newer. If you get errors about `<expected>` or `std::print` on Clang, you're probably on Clang with `libstdc++` instead of `libc++`. The repo's CMake adds `-stdlib=libc++` for Clang; make sure you didn't override it.

(pause)

ASK: "Thumbs up reaction when ctest is green. Type 'stuck' in chat if not." Expect most thumbs within two minutes. For anyone stuck: "Pair up with someone who's green and share their screen for the exercise, or use Compiler Explorer. Don't debug your environment during the lecture; I'll stay after."

DO: If more than a couple are stuck, read the first error from chat and give the one-line fix, then move on at 0:08 regardless.

Okay. Let's look at the program you'll be living with for five sessions.

6. The program we will modernize all course • demo • target 0:08, ~2 min

This is the telemetry record processor in `exercises/s01-modernize-syntax/starter`. It reads lines like the one on the slide: a timestamp in milliseconds, a sensor name, a value, and an optional status. It validates each line against a sensor table, things like "rpm must be between 0 and 12,000," computes per-sensor statistics, and prints a report.

Let's run it.

DO: From the repo root:
`build/exercises/s01-modernize-syntax/s01_starter`
`data/sample.csv`. If "No such file", run it from the exercise folder: `cd exercises/s01-modernize-syntax && ../../build/exercises/s01-modernize-syntax/s01_starter data/sample.csv`. Expect a list of rejected lines, seven of them, one per rejection reason, then a stats block per sensor. Recovery: if the binary is missing, say "it's built by the default target; we'll see it in the exercise" and just open the source.

Look at the top of the output. Seven rejections, and each one is a different reason: malformed timestamp, unknown sensor, out of range, bad status, and so on. The sample data was built to exercise every path. Then the stats per sensor.

DO: Open `starter/record.h`. Scroll slowly past the six comparison operator declarations. Say nothing about them.

(beat)

I want to be clear about this program: it's good C++11. Nothing in it is wrong. It's the kind of code that passes review on most teams today. It's careful about `const`, it doesn't leak, it has tests.

And by the end of today, a lot of it will be gone anyway. Not because it was broken. Because something shorter, safer, or faster became possible. That's the whole course in one sentence.

The tests are how we prove we didn't change behavior. Every exercise in this course runs the same test suite before and after.

(pause)

Before we modernize anything, let's make sure we agree on what C++11 actually means.

7. What this course assumes you know • target 0:10, ~1.5 min

This is the baseline. C++11: `auto`, range-based `for`, lambdas, rvalue references and `std::move`, `unique_ptr` and `shared_ptr`, `nullptr`, `enum class`, `override`, the original `constexpr`, uniform initialization, `static_assert`, and basic threads and atomics.

Quick self-check. I'm going to name three, and you react with a thumbs up if you'd be comfortable explaining it to a new hire at a whiteboard.

ASK: "Thumbs up if you could explain to a new hire: (1) what `auto` deduces from a function that returns a const reference. (2) What `std::move` actually does at runtime. (3) When a lambda capture by reference becomes a bug." Expect fewer thumbs on the second. Say: "Good. That's honest. The next three slides are exactly those three."

(pause)

The next eight slides are the parts people think they know. That's not a dig. These are the C++11 features where the mental model most people carry is slightly off, and slightly off is where bugs live.

If you're thinking "I'll just tune out for twenty minutes," don't. Three of these come back in Session 2 as the foundation for vocabulary types.

(beat)

Let's start with `auto`.

IF BEHIND: "You know the C++11 checklist; the next slides are the parts that bite."

8. `auto` drops references and `const` • C++11 • target 0:12, ~3.5 min

The problem line says it: `auto` deduces like a template parameter. Pass something by value into a template, and the top-level `const` and the reference get stripped. `auto` does the same thing.

Look at the first block on the slide, line 1. `c.get()` returns a `const std::string&`. But `auto a` is a plain `std::string`. It's a copy. The ampersand and the `const` are gone.

auto on its own always means "give me a copy." If the thing is a hundred-byte string, you just allocated and copied a hundred bytes, silently. Put that inside a loop over a map, for (auto p : my_map), and you copy every key-value pair on every iteration.

Line 2 is what you usually meant: `const auto& b`. Now it's a const reference to the original.

Line 3, `auto& d = c.name`. That's a mutable alias straight into the object.

DO: Switch to `auto_pitfalls.cpp` on the right, then terminal:

```
./build/demos/s01/demo_s01_auto_pitfalls. Expect cfg cfg! cfg! 42
2 3 2.
```

Look at the output. Line 20 in the file does `d += "!"`, mutating through the alias. `a` still says `cfg` because it's a copy taken before. `b` says `cfg!` because it's a reference and it sees the change. So `a` isn't just slower, it's stale.

(pause)

The rule: `const auto&` when you're reading, `auto&` when you're mutating, plain `auto` only when you want a copy. And in code review, plain `auto` on a range-for over a container of strings or structs is worth a comment every time.

Second block. Braces. `auto x{42}` is an `int`. In the original C++11 wording it was an `initializer_list<int>`, which surprised everybody, and that was fixed. `auto y = {1, 2}`, with the equals sign, is still an `initializer_list`. In the output, `y.size()` is 2.

Then the vector pair. `v(3, 7)` is three sevens. `w{3, 7}` is the two values 3 and 7. Output: 3 and 2. Parentheses call the count-and-value constructor; braces prefer the `initializer_list` constructor whenever one exists. We'll come back to that trap twice today.

IF AHEAD: The `auto x{42}` fix was N3922, adopted in late 2014 as a defect report against C++14. Compilers apply defect reports retroactively, so modern GCC and Clang give you `int` even under `-std=c++11`. I checked: both compile a `static_assert` that it's `int` in C++11 mode. So if someone on a C++11 codebase says "auto with braces gives `initializer_list`," they're right per the original standard and wrong per their compiler. Also worth knowing: clang-tidy's `performance-for-range-copy` check flags exactly the `for (auto p : map) copy`, and `performance-unnecessary-copy-initialization` flags `auto a = c.get()` when `a` is never modified. Both are cheap to turn on.

IF BEHIND: "auto strips the reference and const, so it copies; write `const auto&` to read and `auto&` to mutate."

So auto silently copies. Next: a function whose name says "move" that sometimes doesn't.

9. When `std::move` does nothing • C++11 • target 0:15, ~3.5 min • SLOW DOWN

`std::move` is a cast. It doesn't move anything. It casts its argument to an rvalue reference, which *permits* a move. Whether a move actually happens depends on what overload gets picked next. Three cases on this slide, and in each one the code compiles and looks fine.

Case one, at the top. Look at `make()`, lines 2 to 7 on the slide. There's a `#ifdef SHOW_ERRORS` around `return std::move(s)`. That line looks like an optimization. It's the opposite.

When you `return s;` for a local, the compiler is allowed to construct `s` directly in the caller's storage. That's named return value optimization, NRVO. Zero copies, zero moves. If it can't do that, the language already treats the returned local as an rvalue and moves it. You get a move for free.

When you write `return std::move(s)`, the return expression is no longer the name of a local. It's a function call result. That disables NRVO. So you've turned "probably zero operations" into "definitely one move."

DO: Terminal: `g++ -std=c++23 -Wall -Wextra -Wpedantic -Werror -DSHOW_ERRORS -c demos/s01/move_does_nothing.cpp -o /dev/null`.
Expect: error: moving a local object in a return statement prevents copy elision [-Werror=pessimizing-move]. Clang says the same words with [-Werror, -Wpessimizing-move]. Recovery: if the shell can't find the file, you're not at the repo root; just read the message from this script.

Both compilers have that warning in `-Wall` now. Under this repo's `-Werror` it's a build break, which is why the wrong line hides behind `SHOW_ERRORS`.

(pause)

Case two. `cases()`, the `const std::string c`. Then `consume(std::move(c))`.

ASK: "Does that move the string, copy it, or fail to compile?" Take two answers from chat. Expect a split between "moves" and "doesn't compile."

(pause)

It copies. Silently. No warning from either compiler. `std::move(c)` gives you a `const std::string&&`. The move constructor takes a non-const `std::string&&`, so it can't bind. The copy constructor takes `const std::string&`, which can bind to anything, so it wins. You wrote "move," you got a copy, and nothing told you.

Where you'll see this in review: someone marks a member `const` for safety, and later someone else adds a `std::move` of it in a move constructor. It quietly copies forever.

Case three. Move from a non-const `s`, and then call `s.size()`. That's legal. A moved-from standard library object is in a "valid but unspecified" state. Valid means you can destroy it, assign to it, call functions with no preconditions like `size()` or `clear()`. Unspecified means you don't get to rely on what's in it.

DO: `./build/demos/s01/demo_s01_move_does_nothing`. Expect four lines: 100, 100, 100, 0. Point at the last 0.

That last zero is what `libstdc++` happens to do. Nothing guarantees it. Short strings in particular may be copied, not moved, so a moved-from short string can still hold its old value.

After a move, the only things you do with the object are assign to it or let it die.

IF AHEAD: clang-tidy has `performance-move-const-arg`, which catches case two, and `bugprone-use-after-move`, which catches reading a moved-from object in most straight-line code. Neither compiler warns on its own. Also: C++20 and C++23 widened the implicit-move rules (P1825, then P2266), so more return statements move automatically, including returning an rvalue-reference parameter by name. The direction of the language is "stop writing `std::move` on returns." And the NRVO caveat: NRVO is still optional in C++23. It's the unnamed case, `return T{...}`, that became guaranteed in C++17, which is slide 36.

IF BEHIND: "`return std::move(local)` disables elision, `std::move` of a `const` silently copies, and a moved-from object is only good for assignment or destruction."

So moves can silently not happen. Lambdas have the opposite problem: references that silently outlive what they point at.

10. Lambdas that outlive their captures • C++11 • target 0:19, ~2.5 min

The problem line: **capture by reference is a pointer to a stack frame**. If the lambda leaves that frame, the pointer dangles.

Look at `make_logger_bad`, the first function. It builds a local `tag`, then returns a lambda that captures everything by reference with `[&]`. The lambda gets stored in a `std::function` and returned. The function returns, `tag` is destroyed, and the lambda is holding a reference to a dead string.

The second function is identical except for the capture: `[tag]`. Now the closure holds its own copy. Safe.

DO: Switch to the godbolt tab with `make_logger_bad("boom")()`; uncommented and `-fsanitize=address -g`. Expect ERROR: AddressSanitizer: stack-use-after-return. Recovery: if godbolt's ASan prints nothing, the flags are in the assembly pane instead of the Executor pane (each has its own options box); fix that, or just say "without the sanitizer it often prints the right thing, which is the scary part" and move on.

There it is. Stack use after return. And note: no compiler warning. Both compilers accept this cleanly under all our flags.

(pause)

`[&]` isn't evil. Inside `std::for_each`, inside `std::sort`'s comparator, any algorithm that finishes before your function returns, it's perfect. **It becomes a bug the moment the closure is stored:** in a `std::function`, handed to a thread, registered as a callback, put in a queue.

Rule for code review: if the lambda is stored or crosses a thread, every capture must be explicit, and anything by reference needs a lifetime argument. I'd go further than `[=]`. In a member function `[=]` captures `this`, which is a pointer, which can dangle exactly the same way. Spell out the captures.

We'll see the C++14 fix in a few minutes: `init-capture` lets you move state into the closure, so there's nothing left to dangle.

IF AHEAD: The same bug shows up with `std::thread` and detached threads: `std::thread([&]{ use(local); }).detach();`. And with any asynchronous API that takes a callback, including timer and I/O completion handlers.

ThreadSanitizer won't catch the lifetime part; AddressSanitizer will. If your CI only runs unit tests without sanitizers, a nightly ASan job is one of the highest-value additions you can make, and it's mostly a flag change. One more: `[=]` captures this implicitly in C++11 through C++17, and C++20 deprecates that, which is on slide 49.

IF BEHIND: "`[&]` is fine for algorithms that finish before you return; for anything stored, capture explicitly by value."

That's lifetime with lambdas. Now lifetime with ownership.

11. `unique_ptr` vs `shared_ptr`: ownership, not "modern pointer" • target 0:21, ~2.5 min

There's a habit in C++11 codebases where "modern C++" got translated to "use smart pointers," and that got translated to "use `shared_ptr` everywhere because it just works." This slide is about undoing that.

`unique_ptr` is the default. Exactly one owner. With the default deleter it's the size of a raw pointer and the generated code for dereference is identical.

`shared_ptr` is a design decision, not a convenience. Let's be precise about the cost. Every copy does an atomic increment of the reference count, and every destruction does an atomic decrement. There's a control block per object, a separate allocation unless you used `make_shared`. And the big one isn't performance at all: **it makes lifetime a runtime question**. Nobody can tell you, from reading the code, when the object dies.

That's the real failure mode. It works until two subsystems disagree about who releases what, and then you get a cycle that never frees, or a callback holding the last reference to its own owner.

Raw `T*`: still fine. As a non-owning parameter that may be null, it's correct C++. What it must never be is an owner. And `T&`, non-owning and never null, is the best parameter type when it fits.

The pattern from C++14 on: factories return `unique_ptr`, containers hold `unique_ptr`, functions take `T&` or `T*`. `shared_ptr` where ownership is genuinely shared: a cache, a graph, an object kept alive by several asynchronous operations. Break cycles with `weak_ptr`.

ASK: "In chat: in your current codebase, roughly what fraction of smart pointers are `shared_ptr`? Most, about half, or few?" Expect "most" from a few people. Say: "That's common, and it's worth an audit. Each one should have an answer to 'who else owns this?'"

(pause)

IF AHEAD: "Zero overhead" has one honest caveat. On the Itanium C++ ABI, which is Linux on x86 and ARM, a `unique_ptr` passed by value can't go in a register, because it has a non-trivial destructor. So `void f(std::unique_ptr<T>)` costs a bit more at the call than `void f(T*)`. It almost never matters, but if someone in review says

"unique_ptr is literally free," that's the asterisk. Also, the atomics: plan on the reference count using atomic operations even in a single-threaded program. libstdc++ has a fast path for programs it believes are single-threaded, but don't design around it. And shared_ptr being thread-safe means the count is thread-safe, not the pointee.

IF BEHIND: "unique_ptr by default; shared_ptr only when ownership is truly shared, because it turns lifetime into a runtime question."

Quick lap through the C++11 features that need no convincing.

12. The C++11 features everyone actually uses • target 0:24, ~1 min

Five quick ones, and I suspect this room uses all of them.

enum class: scoped, no implicit conversion to int, and you can forward-declare it because the underlying type defaults to int. nullptr: a real null pointer with its own type, so f(0) and f(nullptr) pick different overloads the way you'd hope. override and final: the compiler checks you're actually overriding something. static_assert. And = default and = delete to say what you mean about special members.

If anyone is still writing virtual void f() in a derived class without override, this is your moment to stop. Without override, a signature typo, a missing const, a different parameter type, silently declares a new function instead of overriding. With override, it's a compile error.

(pause)

GCC's -Wsuggest-override and Clang's -Winconsistent-missing-override will find the ones you missed.

(beat)

= delete comes back later today, on the copy elision slide. Next, a feature that started tiny.

IF BEHIND: "You use these already; if you don't use override on every override, start."

13. constexpr in C++11: the original form • C++11 • target 0:25, ~1.5 min

This is one of the few hand-typed blocks, because you can't show a C++11 restriction in a file compiled as C++23.

In C++11, a constexpr function was one return statement. No loops, no local variables, no if. You had recursion and the ternary operator, and that was it.

Look at the first function: factorial as a single ternary with recursion. Legal C++11.

The second one, factorial14, is how anyone would actually write it: a local, a loop, a return. That is not legal C++11. It's legal from C++14.

(pause)

So in C++11, `constexpr` was a bit of a toy. People wrote template metaprograms instead, or giant recursive ternaries, or they just computed the table by hand and pasted it in as a constant array with a comment saying "generated by a script we lost."

(beat)

Plant this flag: `constexpr` went from a toy in C++11 to essentially a second language by C++23. Session 3 walks that whole timeline using the CRC table from the exercise. For now, remember that if you learned `constexpr` in 2012, your model is out of date.

IF AHEAD: C++11 did allow a few things beyond the return: `static_assert`, `typedef` and `using` declarations, and `null` statements. And a C++11 `constexpr` member function was implicitly `const`. That changed in C++14, so code written for C++11 that relied on it can produce a new warning or a `const`-correctness surprise when the flag moves. Both compilers give a clear diagnostic.

IF BEHIND: "C++11 `constexpr` was one return statement; C++14 let it look like a normal function; Session 3 has the rest."

14. Uniform initialization and its one trap • target 0:26, ~2 min

Braces everywhere. That was the C++11 pitch. And mostly it's good.

Look at the code. `a(3, 7)`, parentheses: three sevens. `b{3, 7}`, braces: two elements, 3 and 7. `c{}`: empty. And the last line, `Record r{1, "rpm", 4800.0, Status::Ok}`: aggregate initialization, no constructor needed, members in declaration order.

Two things braces do. **Braces prevent narrowing:** `int x{3.5}` is a compile error, where `int x = 3.5` silently truncates. That alone is a good reason to prefer them. In a codebase with a coding standard that worries about implicit conversions, braces enforce part of it for free.

And braces prefer `initializer_list` constructors, which is the trap. If a type has any constructor taking an `initializer_list`, brace initialization will pick it whenever it possibly can. `vector`, `string`, and anything that wraps them.

ASK: "What's in `std::string s{65, 'x'}?`" Pause for chat. Expect "65 x's."

Answer: "Two characters. 65 converts to 'A', so it's the string Ax. Parentheses give you 65 x's."

(pause)

The rule most teams adopt: braces by default, except when the type has an `initializer_list` constructor and you mean a different constructor. Then parentheses, deliberately.

And CTAD, later today, makes this trap slightly easier to hit, because `std::vector v{3, 7}` with no template argument is also two ints. The same lines live in `demos/s01/ctad.cpp`.

IF AHEAD: Why `std::string s{65, 'x'}` compiles at all: narrowing from `int` to `char` is allowed in braces when the source is a constant expression whose value fits, and 65 fits. If the 65 were a runtime `int` variable, it would be a narrowing error. That

subtlety, "constant expressions that fit are not narrowing," is also why `std::uint8_t b{255}` compiles and `std::uint8_t b{256}` doesn't.

IF BEHIND: "Braces stop narrowing but prefer `initializer_list`, so `vector{3, 7}` is two elements."

15. Calibration takeaway • takeaway • target 0:28, ~2 min

So that's the calibration. Let me say the important part plainly.

Good C++11 is good code. Nothing in the starter program is wrong.

Every slide from here replaces some of it anyway: not because it was broken, but because something shorter, safer, or faster became possible.

That framing matters for how you take this back to your team. Nobody needs to feel bad about C++11 code. A lot of our code lives fifteen or twenty years, through toolchain upgrades that lag the standard, through certification baselines that freeze a compiler version. Code written well for its standard keeps working. The question is just what the next edit should look like.

ASK (if you are at 0:28 or earlier): "For those who built the starter earlier and looked around: what struck you as the ugliest part?" Expect someone to name the six comparison operators in `record.h`, or the `.first/.second` on map inserts. Say: "Both of those are gone by the end of today."

(pause)

Now, the first standard after C++11. Small, but it's the one that made modern C++ comfortable.

IF BEHIND: Say only the two bold lines and go.

16. C++14 in one slide • C++14 • target 0:30, ~1.5 min

C++14 shipped three years after C++11. Its job was to fix the things C++11 got almost right.

No large features. About ten small ones, and you'll use half of them every day: generic lambdas, init-capture, relaxed `constexpr`, `make_unique`. Those four are the headline. It removed exactly one thing from the library: `gets`, the C function that can't be used safely because it has no buffer size. If you teach InfoSec, `gets` is the first slide of the buffer overflow lecture. It's gone.

C++14 is the release where modern C++ became comfortable to write.

(pause)

And it's cheap. The slide says it compiles on every compiler you'll meet, and for anything from roughly the last ten years that's true: GCC 5 and Clang 3.4 had essentially complete C++14. If your codebase is on 11, moving to 14 costs almost nothing and buys the next ten slides.

(beat)

The honest exception in our world is a certified or vendor-supplied toolchain for an embedded target that's frozen older than that. If that's you, the badges tell you exactly what you're giving up.

Let's start with the one you'll use most.

IF BEHIND: "C++14 is ten small fixes to C++11, essentially free to adopt; here are the ones that matter."

17. Generic lambdas • C++14 • target 0:32, ~3.5 min

Left side is the C++11 way, and it's straight from the exercise starter's `stats.cpp`. A struct called `ValueDescending` with a `const operator()` that takes two `Records` and compares `.value` with greater-than. Then you pass an instance of it to `std::sort`. Nine lines to say "sort by value, biggest first." And the comparator lives far away from the sort that uses it.

C++11 did have lambdas, so you could write that inline. But C++11 lambda parameters had to be concrete types: `const Record& a, const Record& b`. Fine for one type. If you wanted the same comparator for two types, you were back to a struct with a template `operator()`.

Right side. Look at the `value_descending` line. **auto in a lambda parameter makes the lambda's operator() a template.** That's all a generic lambda is. The compiler generates the same struct with a template call operator that you'd have written by hand, and it does exactly what the nine lines on the left do.

So `value_descending` works for any type that has a `.value` member: `Record`, the demo's little `Reading` struct, your own types.

DO: Switch to `generic_lambda.cpp`, then
./build/demos/s01/demo_s01_generic_lambda. Expect the three rpm readings printed largest first (4830 4800 4795), then captured by move. Point at the `value_descending` line and at the `std::sort` call in `main`. Recovery: the expected order is in the comment next to the sort.

One lambda, declared once at namespace scope, used like any function object, and it replaces the struct on the left one for one.

The two columns are the same comparison: records by value, descending, and the lambda replaces the struct one for one. In the exercise, task 6, `ValueDescending` becomes that one-line lambda written inline at the `std::stable_sort` call in `stats.cpp`, so the comparison sits next to the sort that uses it instead of in a struct forty lines up. Same behavior, less to read.

(pause)

Rule of thumb: use `const auto&` parameters in generic lambdas unless you have a reason not to. Plain `auto` copies, same as slide 8. `auto&&` is the forwarding version, and you'll want it when you're passing arguments through to something else.

ASK: "Is a generic lambda slower than the hand-written struct?" Expect "no" from most. Say: "Right. It's the same template instantiation the struct would get. A lambda is a struct the compiler writes for you."

IF AHEAD: Two things worth knowing. First, a generic lambda's `operator()` is a template, so the unary-plus trick that turns a captureless lambda into a function pointer, `+[ ](int a){ ... }`, doesn't work on it. You can still convert a captureless generic lambda, but only by naming the target type: `int (*fp)(int) = [ ](auto a) { return a; };` Second, `std::sort` comparators must be a strict weak ordering. `>` is fine. `>=` is a classic bug: it says an element is "less than" itself, which is undefined behavior for `std::sort`, and with `libstdc++` it can actually walk off the end of the range on large inputs. If you see `>=` or `<=` in a comparator in review, flag it. C++20 adds template lambdas, `[ ]<typename T>(const T& a, const T& b)`, for when you need to name the type, and that's Session 3.

IF BEHIND: "auto parameters make a lambda's call operator a template: one lambda, any type."

Generic lambdas fixed the parameter side. The next one fixes the capture side.

18. Lambda init-capture • C++14 • target 0:35, ~2.5 min

Here's the question. In C++11, how do you get a `unique_ptr` into a lambda?

(pause)

You can't capture it by copy, because it isn't copyable. You can capture it by reference, which is exactly the dangling problem from slide 10 if the lambda escapes. People wrapped it in a `shared_ptr` just so it could be copied in, or used `std::bind` tricks. Neither was nice.

Init-capture lets you declare a new variable in the capture list and initialize it with any expression, including `std::move`.

Look at the code. `make_printer` takes a `unique_ptr<std::string>` by value, and returns a lambda with `[ s = std::move(owned) ]`. Read that as: the closure has a member called `s`, initialized by moving from `owned`. The closure now owns the string. When the closure dies, the string dies.

DO: Point at line 19 of `generic_lambda.cpp` and the captured by move output line from the previous run. No rerun needed.

And that's the fix for the escaping lambda. **Move the state into the closure, and there's nothing left to dangle.**

The slide shows two more spellings. `[ n = compute() ]` captures the result of an expression, evaluated once when the lambda is created. `[ &r = *ptr ]` captures a reference under a new name, which is handy when the thing you want is behind a pointer or a member.

IF AHEAD: Two gotchas. A closure that holds a `unique_ptr` is itself move-only, so you can't put it in a `std::function`, which requires copyable callables. That's a real problem with callback registries. C++23 adds `std::move_only_function` for exactly this, and that's Session 2 (note the support matrix: `libc++ 18` doesn't have it yet). Second, init-captures are `const` inside the lambda unless the lambda is `mutable`, same as any by-copy capture. So `[ v = std::move(vec) ] { v.push_back(1); }` doesn't compile until you add `mutable`.

IF BEHIND: "[s = std::move(p)] moves state into a closure, which is how you capture move-only types and avoid dangling."

And notice make_printer returns auto. That's the next slide.

19. Return type deduction and decltype(auto) · C++14 · target 0:38, ~2.5 min

The code line at the top is make_printer again, abbreviated with an ellipsis. It returns auto.

In C++14, a function's return type can be auto, and the compiler takes it from the return statements. Every return must deduce the same type, or it's an error. That's the full rule.

Why does it exist? Because some types can't be named. A lambda's type is a unique, unnamed class the compiler makes up. If make_printer had to spell its return type, you'd be stuck wrapping it in a std::function, which costs a heap allocation and an indirect call. With auto, you return the closure itself, at zero cost.

Then decltype(auto). Plain auto deduces like auto always does: it strips references and const, slide 8 again. **decltype(auto) deduces exactly, references included.** You want that in a forwarding wrapper: a function that calls another function and should return exactly what that function returns, reference and all. It's rare. If you see it outside a generic wrapper, ask why.

And the last bullet is the one that matters for teams. **Don't use auto return types in public headers.** The signature is the documentation. If a colleague has to read your function body to know what it returns, the interface is worse. Also, an auto function can't be called until its definition has been seen, so it has to live in the header, which defeats separate compilation.

ASK: "Where would you allow auto returns in your codebase's coding standard?"
Expect "local helpers, lambdas, templates." Say: "That's the consensus. Explicit types on anything a colleague calls; auto for local helpers, generic code, and anything returning a lambda."

(pause)

IF AHEAD: The classic decltype(auto) trap: decltype(auto) f() { int x = 0; return (x); }. The parentheses turn the name into an expression, decltype((x)) is int&, and you've returned a reference to a local. Both compilers warn under -Wall (-Wreturn-local-addr on GCC, -Wreturn-stack-address on Clang), so -Werror saves you. Another: recursion works with auto returns as long as a non-recursive return statement appears first, because the type has to be known before the recursive call.

IF BEHIND: "auto returns are for lambdas and local helpers; keep explicit types in public headers."

Now the library addition that made a coding rule enforceable.

20. `std::make_unique` • C++14 • target 0:40, ~2.5 min

`make_shared` shipped in C++11. `make_unique` was forgotten, and it arrived in C++14.

Left side, top. `std::unique_ptr<Sensor> s(new Sensor("rpm", 12000))`. The type appears twice, and there's a `new` in your code. Right side: `auto s = std::make_unique<Sensor>("rpm", 12000)`. The type appears once. No `new`.

With `make_unique`, "never write new" became a coding rule you can actually enforce. `clang-tidy's` `modernize-make-unique` will rewrite the left into the right across a codebase. And once the rule is "no naked new," every remaining `new` in review is a question someone has to answer.

Now the lower half. This is the exception-safety story, and it's subtle.

```
f(std::unique_ptr<A>(new A), std::unique_ptr<B>(new B)).
```

ASK: "Can that leak in C++11?" Pause for chat. Expect a split.

(pause)

In C++11 and 14, yes. The compiler was allowed to interleave the argument evaluations: do `new A`, then `new B`, then construct both `unique_ptr`s. If `new B` throws, `A` was allocated but no `unique_ptr` owns it yet. Leak.

C++17 closed that hole. Each function argument is now evaluated completely before the next one starts. The order between arguments is still unspecified, but they can't interleave. So in C++17 the left version can't leak anymore. Slide 40 comes back to that.

But the right side is still better: shorter, no `new`, the type said once.

IF AHEAD: `make_unique` has limits. It can't take a custom deleter, so for a `C` handle with a `close` function you still construct the `unique_ptr` directly. For arrays, `make_unique<T[]>(n)` value-initializes, which zeroes the memory. That can matter for a large buffer you're about to overwrite, so C++20 added `make_unique_for_overwrite`, which default-initializes. And unlike `make_shared`, `make_unique` gives no allocation advantage. `make_shared` combines the object and the control block in one allocation; `make_unique` has no control block to combine.

IF BEHIND: "make_unique arrived in C++14, says the type once, and lets you ban naked new."

Two small ones that embedded engineers tend to love.

21. Binary literals and digit separators • C++14 • target 0:43, ~1.5 min

Look at the first line. `0b0001'0000'0010'0001`. That's `0x1021`, the CRC-16 CCITT polynomial, and now you can see the bits. That line is from the exercise solution's `crc.cpp`.

Binary literals with `0b`, and the single quote as a digit separator, in any base. The compiler ignores the quote completely. It's for humans.

Next lines: `12'000.0` for the rpm maximum, which is in the exercise's sensor table.
`86'400'000` milliseconds per day, which you can actually read. And the hex mask grouped by bytes, `0xFF'FF'00'00`.

For register masks and bit fields, this is a genuine readability win, and anything that makes a mask easier to check in review is a safety feature.

(pause)

IF AHEAD: The separator can go anywhere between digits, not only every three. So group binary by nibble or by register field, like `0b1'011'0000` for a 1-bit, 3-bit, 4-bit layout, and the grouping documents the layout. Before C++14, GCC supported `0b` as an extension, so older code may have it already; -Wpedantic under C++11 flags it.

IF BEHIND: "0b binary literals and ' separators: the compiler ignores the quote, the reviewer doesn't."

From literals for numbers to literals for types.

22. Standard literals • C++14 • target 0:44, ~3 min

C++11 let you define your own literal suffixes. C++14 shipped a few in the standard library.

ASK: "Quick one: what's the type of `auto x = "abc";`?" Expect "const char*", maybe "std::string" from one or two. Say: "const char*. A string literal is an array of const char, and auto decays it to a pointer. Which is the reason for line 1."

(pause)

Look at line 3. "temp_core"s, with an s suffix, is a `std::string`. That's how you get a real string with auto.

But **the chrono literals are the ones that change code**. `250ms` is a `std::chrono::milliseconds`. `2h + 30min` is `std::chrono::minutes`, because chrono picks the finer unit when you add. And `sleep_for(100us)`.

Here's why it matters. `sleep_for(100)` doesn't compile. There's no implicit conversion from an integer to a duration. So the unit is always in the source. Compare that to a C API where the argument is `int timeout` and you have to look it up to find out whether it's milliseconds, microseconds, or ticks. Mixing units across an interface is one of the most common integration bugs there is, and chrono with literals turns it into a compile error.

Convention: using namespace std::literals; inside a function or at file scope in a .cpp. Never in a header. A using-directive in a header pushes those names into every file that includes it.

IF AHEAD: The chrono conversion rules are worth one sentence each. Converting to a finer unit is implicit and exact: `milliseconds ms = 2s;` is fine. Converting to a coarser unit that would lose precision is a compile error: `seconds s = 2500ms;` won't compile; you need `duration_cast`, or in C++17 `floor`, `ceil`, or `round`, which say how to round. That's exactly the discipline you want for timing code. And

`std::literals` pulls in all three families: `string`, `chrono`, and `complex`. If you only want one, using `namespace std::chrono_literals;`

IF BEHIND: `"abc"s` is a `std::string` and `100ms` is a duration; chrono literals put units in the source so unit mix-ups don't compile."

Now the first attribute most people put in their own code.

23. `[[deprecated]]` • C++14 • target 0:47, ~2 min

The code shows an old parser signature: `parse_record(const char* line, std::size_t len, Record* out)`, marked `[[deprecated("use parse_record(std::string_view) instead")]]`.

Every call site now gets a compiler warning carrying your message. And under `-Werror`, that's a build break, which sounds bad, until you realize it's a build break you schedule.

Here's the migration pattern. You add the new signature. You put `[[deprecated]]` on the old one. Under `-Werror`, every caller lights up, so either you fix them in the same change, or you temporarily allow that one warning with `-Wno-error=deprecated-declarations` while teams migrate over a sprint. Then you delete the old one. **It's much better than a `grep`, because the compiler finds every caller, including the ones in code you didn't know about.**

(pause)

It works on functions, types, variables, typedefs, and template specializations. C++17 extended it to enumerators and namespaces.

And note the syntax: double square brackets. That's the general attribute syntax from C++11, and today we'll put `[[nodiscard]]`, `[[fallthrough]]`, `[[maybe_unused]]`, `[[likely]]`, and a few more in the same place.

(beat)

IF AHEAD: Compilers ignore attributes they don't know, usually with a warning, `-Wattributes` on GCC and `-Wunknown-attributes` on Clang. Under `-Werror` that warning becomes a build break, which is why a newer attribute in a header shared with an older compiler can break that older build. That's the `[[assume]]` story on slide 51. The portable guard is `__has_cpp_attribute(deprecated)`, which you can test in `#if`.

IF BEHIND: `[[deprecated(\"reason\")]]` makes the compiler find every caller for you; it's the migration tool."

24. Relaxed `constexpr` • C++14 • target 0:49, ~2 min

This is an evolution slide. One line per standard, and you'll see this format again in Session 3.

Today, only the C++14 line matters: `constexpr` functions can now look like normal functions. Loops, local variables, `if`, mutation of locals, multiple returns. The `factorial14` from slide 13 is legal from here on.

Let me read the rest of the timeline so you see the shape. C++17: `constexpr` lambdas and `if constexpr`. C++20: dynamic allocation inside a constant evaluation, which means `std::vector` and `std::string` can be used at compile time, as long as the memory is freed before the evaluation ends; plus virtual calls, `try` blocks, and two new keywords, `constexpr` and `constinit`. C++23: most remaining restrictions gone, including `constexpr` `unique_ptr`.

(pause)

The direction is clear: each standard makes "runs at compile time" less of a separate language and more of a property of ordinary code.

Session 3 walks this whole timeline with the CRC table from the exercise: you'll compute the 256-entry table at compile time instead of pasting it in.

IF AHEAD: One thing people miss: `constexpr` on a function means "can be evaluated at compile time," not "will be." If you call it with runtime arguments, it runs at runtime like any function. If you need a guarantee, you assign the result to a `constexpr` variable, or in C++20 mark the function `constexpr`. That distinction matters when someone says "it's `constexpr` so it's free." It's free only where it's used in a constant expression.

IF BEHIND: "From C++14, `constexpr` functions can have loops and locals; Session 3 tells the rest of this story."

25. Small library additions • C++14 • target 0:51, ~2 min

This is a name-drop slide, so I'll move fast, and stop on one.

`std::exchange` is the one worth a sentence. It sets an object to a new value and returns the old one. The idiom is the move constructor: `ptr_ = std::exchange(other.ptr_, nullptr);`. One line: take the pointer, null out the source. Before, that was a temp variable and two assignments, and the "null out the source" step is exactly the line people forget, which gives you a double free.

(pause)

The rest, quickly. `integer_sequence` and `index_sequence`: the tool for unpacking a tuple into function arguments in variadic templates. Free `std::cbegin`, `std::cend`, `std::rbegin`, `std::rend`. `std::quoted`, for streaming strings with quotes and escapes. `std::shared_timed_mutex`, a reader-writer lock, which we'll see in Session 5.

And transparent comparators. `std::map<std::string, T, std::less<>>`. With `std::less<>`, the map can look up by a `const char*` or a string view without constructing a temporary `std::string` for every lookup. That matters once `string_view` arrives in Session 2.

(beat)

IF AHEAD: The transparent comparator only helps `find`, `count`, `lower_bound`, `upper_bound`, `equal_range`, and in C++20 `contains`. Insertion still needs a real

key. And `unordered_map` didn't get heterogeneous lookup until C++20, where you need both a transparent hash and a transparent equality. For a hot lookup path keyed by string, switching to `std::less<>` is a one-token change that removes an allocation per lookup.

IF BEHIND: "std::exchange for move constructors; the rest is reference material."

26. C++14 takeaway • takeaway • target 0:53, ~2 min

C++14, summed up. Removed: `gets`. Added: nothing large.

What changes for you: lambdas become the default way to write a function object, `make_unique` makes `new` disappear, and `constexpr` becomes something you can actually write.

Monday morning: turn on clang-tidy's `modernize-make-unique` and `modernize-use-auto`. Both can apply fixes automatically, and both are low risk. Run them on one directory, read the diff, and see what you think.

(pause)

One caution on `modernize-use-auto`: by default it only rewrites where the type is already obvious on the same line, like after `new` or a cast, or for iterators. It won't turn every declaration into `auto`, which is what you want, given slide 8.

DO: Time check. You should be at 0:53 to 0:55. If you're past 0:57, slide 40 can be skipped with its IF BEHIND line, and slide 33 takes ten seconds.

(beat)

That's C++14, the polish. Now C++17, which is the release you'll feel in every function.

IF BEHIND: Say only the two bold lines.

27. C++17 in one slide • C++17 • target 0:55, ~1 min

C++17 is the release that changed how ordinary functions look.

Language side: structured bindings, `if` with an initializer, `if constexpr`, inline variables, fold expressions, class template argument deduction, guaranteed copy elision, and the useful attributes. Library side: `optional`, `variant`, `string_view`, `filesystem`, parallel algorithms, `from_chars`. The types are Session 2. Today is the syntax.

And it removed things: `auto_ptr`, the `register` keyword, trigraphs, dynamic exception specifications, `random_shuffle`.

This is the segment you'll use most tomorrow. Fifteen slides, twenty-five minutes. **If you only remember two things from it, make them structured bindings and `if` with initializer.**

(pause)

Here's the first one.

IF BEHIND: "C++17 is the everyday release; structured bindings and if-with-initializer are the two to keep."

28. Structured bindings • C++17 • target 0:56, ~2.5 min

Left side, C++11. This is the pattern from `compute_stats` and `load_stream` in the starter, which is exercise task 3.

`map::insert` returns a pair: an iterator to the element, and a bool saying whether it was inserted. In C++11 you write out the whole pair type, `std::pair<std::map<std::string, int>::iterator, bool>`, and call the result `r`. Then you have `r.second` and `r.first->second`.

ASK: "Without looking back at the declaration: in `++r.first->second`, which `second` is the bool?" Expect a slow answer or "neither." Say: "Neither. `r.second` is the bool; `r.first->second` is the `map`'s value. And that's the problem: `.first` and `.second` carry no meaning."

(pause)

.first and .second are the smell. Right side: `auto [it, inserted] = counts.insert({key, 1})`. Now the halves have names. `it` and `inserted`. The `if` reads `!inserted`, and the increment reads `++it->second`. You can tell what it does from the names.

And look where the binding is: inside the `if`, before a semicolon. That's `if` with an initializer, the slide after next. `it` and `inserted` exist only for that `if` statement.

DO: Switch to `structured_bindings.cpp`. Terminal:
`./build/demos/s01/demo_s01_structured_bindings`. Expect `a=2` and `b=1`.
Point at lines 33 to 35: "a" inserted by the C++11 version, then incremented by the C++17 version, so both versions do the same job. Point at line 36: the `map` loop with `[key, n]`, which is the next slide.

A structured binding gives names to the pieces of something that already has pieces.
That's the whole feature.

IF AHEAD: What the compiler actually does: it creates one hidden variable holding the whole result, and `it` and `inserted` are names that refer into it. They're not separate variables. Consequence: the `auto` and any `const` or `&` apply to the hidden object, not to each name. So `const auto& [a, b] = f();` binds a `const` reference to the returned temporary, extending its lifetime, and `a` and `b` refer into it. Another consequence: before C++20 you couldn't capture a structured binding in a lambda; C++20 allows it.

IF BEHIND: "`auto [it, inserted] = map.insert(...)` replaces `.first` and `.second` with names."

29. Structured bindings: what they bind · C++17 · target 0:59, ~2 min

Four things you can bind to. A `std::pair`, like `value_range` returning low and high. Any struct whose non-static members are all public, like the result of `from_chars`, which is a struct with `ptr` and `ec`. A C array, like a `double[3]` point. And tuples.

But the last line is the reason the feature exists. **for (const auto& [name, stats] : by_sensor). The map loop, finally readable.** Compare that to `for (const auto& p : by_sensor)` and then `p.first` and `p.second` in the body. Every map loop in your codebase gets better.

Three rules at the bottom.

It always introduces new names. You can't bind into existing variables. If you want to assign into variables you already have, `std::tie(a, b) = f();` from C++11 still exists for that.

`auto&` and `const auto&` avoid copying the whole object. Same rule as slide 8. `auto [name, stats]` in a map loop copies every pair.

(pause)

And the count must match. Two names on a struct with three members is a compile error. There's no "skip this one."

(beat)

IF AHEAD: Two things people try that don't work. You can't bind to a struct with private members, or one whose members are split between a base and a derived class. And there's no way to ignore an element in C++17 or 23. People write `[[maybe_unused]] auto [a, unused] = ...` to quiet the unused warning, which works. C++26 adds `_` as a placeholder name for exactly this. Also, you can make your own class work with structured bindings by specializing `std::tuple_size`, `std::tuple_element`, and providing `get`. That's how `std::array` and `std::tuple` support it.

IF BEHIND: "Bindings work on pairs, tuples, arrays, and public structs; always new names; use `const auto&` in map loops."

30. if and switch with initializer · C++17 · target 1:01, ~2 min

This is the syntax that showed up inside the `if` two slides ago.

if (init; condition): the same scoping as the init-statement of a for loop. The variable lives for the whole `if` and its `else` chain, and no longer.

First example: `try_emplace` into counts, bind `it` and `inserted`, test `!inserted`. After the closing brace, `it` and `inserted` are gone.

Why does that matter? **The bug it prevents: a variable declared before the if leaks into the rest of the function, gets reused fifty lines later, and by then it means something**

else. Think of a status or an iterator that gets checked once and then, much later, gets checked again after the thing it pointed into has changed. Scoping it to the `if` makes that impossible.

Second line is my favorite. `if (std::lock_guard lk{mu}; queue.empty())`
`{ return; }`. The lock is taken in the init statement, held for the condition and the whole `if/else`, and released before the next statement. That's a precise critical section with no extra braces. And note `std::lock_guard lk{mu}` with no template argument. That's CTAD, slide 37.

Third: `switch` gets the same thing. `switch (auto status = poll(); status.kind)`. Poll once, switch on a field, and `status` is visible in every case.

(pause)

IF AHEAD: One subtlety for the lock example: the lock is held for the `else` branch too. If the `else` does something slow, like I/O, you're holding the lock across it. That's the same rule as any scoped lock, but the compact syntax makes it easier to miss in review. Also, C++20 adds the same init-statement to range-based `for`, and that's slide 46, where it fixes a real dangling bug.

IF BEHIND: "`if (init; cond)` scopes the variable to the `if/else`, which stops stale variables leaking down the function."

31. The map API C++11 should have had • C++17 • target 1:03, ~2 min

Four new map operations.

Line 1: `try_emplace(name)`. Constructs the value only if the key is new. Line 2: `insert_or_assign`: insert or overwrite, and the returned `bool` tells you which happened. Line 3: `extract` and `insert` a node, which moves an element from one map to another without reallocating or copying the element. Line 4: `merge`, which splices every element from `b` into `a` whose key doesn't already exist in `a`.

try_emplace is the one you'll use. The starter's `compute_stats` does `insert(make_pair(name, SensorStats()))`. That builds a `SensorStats` on every call, even when the key already exists and the new value is thrown away. `try_emplace` builds it only if it's actually going in. That's the bonus in exercise task 3.

The second bullet is the subtle one. C++11's `emplace` could also construct and then discard. And worse, it could move from your arguments even when the key already existed, because an implementation is allowed to build the node first and then look for the key. **try_emplace guarantees it never touches your arguments unless it inserts.** So if you passed `std::move(big_thing)`, `big_thing` is still intact after a failed insert.

ASK: "What does `map[key] = value` do that `insert_or_assign` doesn't tell you?" Expect "whether it existed." Say: "Right, and `operator[]` also requires the value type to be default-constructible, because it default-constructs first and then assigns. `insert_or_assign` doesn't."

(pause)

IF AHEAD: `extract` is more useful than it looks. It's the only way to change a key in a `std::map` without a delete and a reallocation: extract the node, modify `node.key()`, insert it back. And a node handle can move between maps with the same key and value type, even with different comparators. For long-lived systems that care about heap churn, that's a real tool.

IF BEHIND: "`try_emplace` only constructs if it inserts and never steals your arguments; use it instead of `insert(make_pair(...))`."

32. inline variables • C++17 • target 1:05, ~2 min

Left side: the C++11 dance. In the header, `extern const std::size_t kMaxLineLength;`. In exactly one `.cpp`, the definition with the value. Or a static copy in every translation unit. Or a function that returns a static. The starter's `config.h` does the extern-and-definition pair, which is exercise task 7.

Right side: `inline constexpr std::size_t kMaxLineLength = 256;`. In the header, and nowhere else.

inline on a variable means what it has always meant on a function: it may be defined in many translation units, and they all refer to the same single object. The linker keeps one.

And the struct below: `static inline const std::string kUnits = "raw";`. Static data members can be defined inside the class now. No out-of-class definition in a `.cpp`. That used to be a constant source of "undefined reference" link errors.

(pause)

What this buys you: header-only libraries can have globals. And the cleanest singleton you'll ever write is `inline Registry g_registry;` in a header.

(beat)

IF AHEAD: The fine print, which an attentive reviewer will ask about. A namespace-scope `const` or `constexpr` variable already had internal linkage in C++11, so `constexpr std::size_t kMax = 256;` in a header already compiled fine. You got one copy per translation unit, which for an integer is harmless. The difference shows up when the address matters, or when an `inline` function in the header odr-uses it: then each translation unit sees a different object, which is technically an ODR violation. `inline constexpr` gives you exactly one object. Also, since C++17, `static constexpr` data members are implicitly `inline`, so the old out-of-class definition is now redundant. Compilers accept it, with a deprecation note in the standard.

IF BEHIND: "`inline constexpr` in the header replaces the extern declaration plus `.cpp` definition; one object, defined once, everywhere."

33. Nested namespace definitions • C++17 • target 1:07, ~0.5 min

Thirty seconds. namespace telemetry { namespace detail { namespace crc { and three closing braces, versus namespace telemetry::detail::crc {. **Purely cosmetic, universally adopted.**

(pause)

C++20 lets you put inline in the middle for versioned namespaces. Clang-tidy's modernize-concat-nested-namespaces does the rewrite for you.

(beat)

Next is the attribute that finds real bugs.

IF BEHIND: Say only "Nested namespaces in one line" and advance.

34. [[nodiscard]] • C++17 • target 1:07, ~2.5 min

The problem line: **a function whose return value is the whole point of calling it shouldn't compile silently when the value is dropped.**

Line 1: [[nodiscard]] bool parse(...). Call parse and ignore the bool, and the compiler warns. Line 2 is the C++20 form with a reason string: "released immediately if dropped", which shows up in the diagnostic. Line 3 is the clever one: struct [[nodiscard]] Error. Put it on a type, and every function returning that type is nodiscard automatically. For an error type, that's exactly what you want.

DO: Terminal: g++ -std=c++23 -Wall -Wextra -Wpedantic -Werror -DSHOW_ERRORS -c demos/s01/nodiscard.cpp -o /dev/null. Expect two errors: ignoring return value of 'bool parse(const std::string&, int*)', declared with attribute 'nodiscard' [-Werror=unused-result] on line 39, and ignoring returned value of type 'Error', declared with attribute 'nodiscard' on line 40. Recovery: read them from this script.

Look at the second error. try_write() itself isn't marked. The type is. That's the type-level version doing its job.

ASK: "Where in your codebase would adding [[nodiscard]] find a bug tomorrow?" Expect "error-code returns," "status checks," "empty()." Say: "All three. And here's a classic: v.empty(); written when someone meant v.clear();. the standard library has marked empty() nodiscard since C++20, so that one is already caught."

(pause)

Put it on parsers, lookups, anything returning an error or a handle, empty(), and factories. Leave it off functions whose side effect is the point. printf returns a count nobody reads, and std::map::insert returns a pair you often don't need. Marking those would be noise.

This is the attribute that finds real bugs in existing code, the day you add it. Exercise task 4 asks you to add it across the starter and read what the compiler says.

IF AHEAD: Defense codebases often already have a rule that return values are checked or explicitly discarded, because MISRA and CERT-style standards require it. `[[nodiscard]]` turns that rule from a static-analysis finding into a compiler error. When a discard really is intended, the idiom is `static_cast<void>(parse(...));`, or in C++26, `auto _ = parse(...);` with the new placeholder name. Also: `[[nodiscard]]` on a constructor, which C++20 allows, flags `Guard{mtx};` written without a variable name, which constructs and immediately destroys the guard. That's one of the nastiest lock bugs there is.

IF BEHIND: "`[[nodiscard]]` on functions or types makes ignoring the result a compile error under `-Werror`; put it on anything returning an error, handle, or lookup."

35. `[[maybe_unused]]` and `[[fallthrough]]` · C++17 · target 1:10, ~1.5 min

Two warnings you used to silence with tricks the compiler couldn't read.

Look at the `verbosity` function. The parameter `color` is marked `[[maybe_unused]]`. Imagine it's only used in some builds, behind an `#ifdef`. Before C++17 you'd write `(void)color;` somewhere in the body. Now the declaration says it.

Then the `switch`. `case Level::Trace:` prints, and then deliberately falls into `case Level::Debug`. `[[fallthrough]]`; says **"this fall-through is intentional," in a way the compiler checks**. It must be immediately before a case label, or it's an error.

On GCC, `-Wextra` turns on `-Wimplicit-fallthrough`, so any fall-through without the attribute becomes an error under `-Werror`. On Clang, `-Wextra` doesn't include it; add `-Wimplicit-fallthrough` yourself. Either way, **a forgotten break is now a compile error, and an intended fall-through is documented**.

(pause)

A forgotten break is one of the oldest bugs in C, and it shows up in every coding standard for a reason.

(beat)

IF AHEAD: GCC also accepts a `// fall through` comment as the marker, at the default `-Wimplicit-fallthrough=3` level. That's why some older codebases have strange comment conventions. The attribute is the portable form. `[[maybe_unused]]` also applies to local variables, functions, types, and enumerators, which is handy for something only used inside an `assert`, which disappears under `NDEBUG`.

IF BEHIND: "`[[fallthrough]]` marks deliberate fall-through, `[[maybe_unused]]` replaces the void cast."

36. Guaranteed copy elision • C++17 • target 1:11, ~3 min • SLOW DOWN

This one changes the model in your head, so I'm going to slow down.

The problem line: before C++17, returning by value required a copy or move constructor to *exist*, even when the compiler was going to elide the call and never run it.

Look at `Pinned`, the struct on the slide. It has a `std::mutex` member, so it can't be copied or moved. And it explicitly deletes its copy constructor, which also suppresses the implicit move constructor. It's pinned in memory.

Now `make_pinned`: `return Pinned{id};`. And in `main`, `Pinned p = make_pinned(7);`.

ASK: "In C++14, does `make_pinned` compile?" Expect mixed answers. Wait for three.

(pause)

DO: Godbolt tab with `copy_elision.cpp`. Change `-std=c++23` to `-std=c++14`. Expect error: use of deleted function '`Pinned::Pinned(const Pinned&)`' on line 17 and again on line 21. Change it back to `-std=c++17`: compiles, prints 7. Recovery: if godbolt is slow, run `g++ -std=c++14 -c demos/s01/copy_elision.cpp -o /dev/null` locally; same error.

In C++14, no. Even though every compiler would have elided the copy, the rules said the copy constructor had to be callable.

In C++17, a prvalue isn't a temporary object anymore. It's a recipe for initializing an object. `Pinned{id}` in the return statement doesn't create a `Pinned`. It describes how to create one. The recipe gets carried out exactly once, at its final destination, which is `p` in `main`. **No object is ever copied or moved, so no copy or move constructor needs to exist.**

(pause)

What this buys you. **Factory functions for types that can't move:** mutexes, atomics, objects that represent a hardware register block or a handle that must stay at one address. And `T x = T(args);` is now exactly the same as `T x(args);`. No hidden temporary.

Now connect it back to slide 9. The guarantee covers returning a prvalue, an unnamed object like `Pinned{id}`. **Returning a named local is still NRVO, and NRVO is still optional.** That's why `move_does_nothing.cpp` says "usually elided." And that's why `return std::move(local)` is a pessimization: it turns an elidable name into an expression that has to be moved.

IF AHEAD: In standardese, the change was P0135, "guaranteed copy elision through simplified value categories." The deeper idea is "temporary materialization": a prvalue only becomes a real object when it has to, for example when you bind a reference to it or call a member function on it. That's also why Session 2's `optional` and `variant` can hold non-movable types via in-place construction. One subtle limit: `return cond ? Pinned{1} : Pinned{2};` is still fine, a conditional of prvalues is a prvalue. But `Pinned a{1}; return a;` is not, because `a` is named, and

Pinned can't be moved. There's a supplemental deck on value categories if anyone wants the full taxonomy.

IF BEHIND: "In C++17, returning an unnamed object constructs it directly in the caller, so non-movable types can be returned from factories; named locals are still only usually elided."

37. Class template argument deduction • C++17 • target 1:14, ~1.5 min

Problem line: you had `std::make_pair` because you couldn't write `std::pair` without the template arguments.

CTAD: the compiler deduces a class template's arguments from the constructor arguments, the same way it always did for function templates. `std::pair p{1, 2.5}` is a `pair<int, double>`.

The killer use is line 2: `std::lock_guard lk{m};`. Nobody misses typing `std::lock_guard<std::mutex>`. Line 3: `std::vector v{1, 2, 3}` is a `vector<int>` with three elements.

And then the trap, lines 4 and 5, same as slide 14. `std::vector<int> a(3, 7)` is three sevens. `std::vector b{3, 7}` is two ints. CTAD makes the trap easier to hit because there's no `<int>` to remind you that you're choosing a constructor.

(pause)

DO: `./build/demos/s01/demo_s01_ctad`. Expect `1 2.5 3 3 2`. Point at the last two numbers: `a.size()` is 3, `b.size()` is 2.

(beat)

IF AHEAD: You can write your own deduction rules, called deduction guides, but most code never needs them. The one trap to know: `std::vector v2{v}` where `v` is a vector gives you a copy, a `vector<int>`, not a vector of vectors. The copy deduction candidate wins. That's usually what you want, but it surprises people writing generic code. And some teams ban CTAD in public APIs because the type isn't visible at the declaration; `-Wctad` - maybe - unsupported on both compilers flags CTAD on templates whose authors didn't intend it.

IF BEHIND: "CTAD deduces template arguments from constructors; `std::lock_guard lk{m}` is the win, `std::vector v{3, 7}` is the trap."

38. `std::from_chars` and `std::to_chars` • C++17 • target 1:15, ~2.5 min

Left side is `parse_value` from the starter's `parser.cpp`, exercise task 5. It uses `strtod`. Count the problems.

`errno = 0`. Global state, and you must reset it first because `strtod` only sets it on failure. `char* end`, and the input must be NUL-terminated, so you need `c_str()`, and if you had a slice of a bigger buffer you'd have to copy it into a string first. And `strtod` is **locale-dependent**. It uses the C locale's decimal separator.

Here's how that bites. If any code in the process calls `setlocale(LC_ALL, "")` and the machine runs in a German locale, the decimal separator becomes a comma. Then `strtod("22.150")` stops at the dot and returns 22, with end pointing at the dot. In this function that's a rejection, and in a sloppier one it's a silently truncated temperature.

Right side. `std::from_chars(first, last, value)`. It takes a range, not a C string. **No `errno`, no NUL terminator, no locale, no allocation, and no exceptions.** It returns a struct with `ptr`, where parsing stopped, and `ec`, an error code. And look: that struct is made for a structured binding. `auto [ptr, ec] = ...` Success means `ec == std::errc{} and ptr == last`, meaning the whole field was consumed.

(pause)

ASK: "One behavioral difference between left and right. What happens to `*out` when the input is `41.25x`?" Expect "it's untouched" from most. Answer: "On the left, untouched: the function returns false before writing. On the right, `from_chars` already wrote 41.25 into `*out`, and then the function returns false because `ptr` isn't at the end. So the right version clobbers the output on failure."

That's worth knowing for the exercise, because the tests will tell you if any caller relies on the output being untouched. If one does, parse into a local and assign only on success.

Two more behavior changes. `from_chars` doesn't skip leading whitespace, and it doesn't accept a leading plus sign. `strtod` accepts both. **So `" +22.1 "` parses with `strtod` and is rejected by `from_chars`.** If your field data can have a plus sign, that's a behavior change, and the tests are how you find out.

The caveat from the slide: floating-point `from_chars` needs `libstdc++ 11` or later, or `libc++ 20`. Clang 18 with `libc++` doesn't have it, so the demo file and the exercise solution fall back to `strtod` under `#ifndef __cpp_lib_to_chars`.

IF AHEAD: Don't run `demo_s01_from_chars` to show this; its `printf` reads `a` and `b` in the same call that parses into them, and the order of function argument evaluation is unspecified. GCC evaluates right to left and prints `0.00 0.00`; Clang evaluates left to right and prints the parsed values. That's a perfect live example for slide 40, if you want it: build it with both compilers and show different output from the same source. `to_chars` is the other direction, and its key property is round-trip: by default it produces the shortest string that reads back to exactly the same double, which `printf("%g")` doesn't guarantee. For telemetry logs that get re-parsed, that matters.

IF BEHIND: "`from_chars` parses a range with no `errno`, no locale, and no allocation; watch that it rejects a leading plus and writes the output even on trailing garbage."

39. `std::string_view`: a preview • C++17 • target 1:18, ~1 min

One slide on this today, because Session 2 does it properly.

A `string_view` is a pointer and a length. It doesn't own anything. So

`parse_record(std::string_view line, ...)` accepts a string literal, a `std::string`, or a slice of a buffer, with no copy and no allocation.

A correction to the second code line. `std::string` doesn't have a `substr_view` member. The way to get a non-allocating slice is to make the view first and slice the view:

`std::string_view(buffer).substr(0, n)`. `std::string::substr` returns a new `std::string`, which allocates.

(pause)

Never store a `string_view` unless you own what it points at. That's the same rule as the escaping lambda: a view is a reference. And it's the same rule as `span` next session. Same bug, three spellings.

IF BEHIND: "A `string_view` is a non-owning pointer and length; never store one; Session 2 covers it."

40. Two quiet fixes • C++17 • target 1:19, ~1 min

Optional if you're behind, but these two make C++11 code subtly wrong and C++17 code correct.

Evaluation order. C++17 fixed the order for a set of expressions. In `a . b` and `a -> b`, the left side first. In `a ( b )`, the function expression before the arguments. In `a = b`, the right side first. In `a << b`, left first, which is why chained `std::cout <<` with side effects is now well-defined.

And separately, **function arguments can no longer interleave.** That's what fixes the `make_unique` leak from slide 20: each argument is fully evaluated, including its new and its `unique_ptr` construction, before another one starts.

What's still unspecified: the order *between* arguments. `f(g(), h())` can call `h` first. If you want a demo, the `from_chars` demo prints different numbers on GCC and Clang for exactly this reason.

A cleaner example of the assignment rule than the one on the slide: `m[k] = m.size();` In C++17, `m.size()` is evaluated first, then `m[k]` inserts. Before C++17 it was unspecified whether the new element was counted.

(pause)

`noexcept` is part of the function type now. `void f() noexcept` and `void f()` are different types. A pointer declared `noexcept` can't hold a function that might throw, and templates can now see and deduce the difference. The practical consequence for long-lived code: function pointer types mangle differently, so mixing C++14 and C++17 objects that pass such pointers across an interface can fail to link. GCC's `-Wnoexcept-type` warns about exactly that.

(beat)

IF BEHIND: "C++17 fixed evaluation order for assignment, member access, shifts, and argument interleaving, and made `noexcept` part of the type."

41. C++17 takeaway • takeaway • target 1:20, ~0.5 min

Removed: `auto_ptr`, `register`, `trigraphs`, dynamic exception specifications, `random_shuffle`, and the `bind1st` family.

Structured bindings and `if-with-initializer` reshape every function; `[[nodiscard]]` finds bugs the day you add it; `inline` variables end the extern-and-definition dance.

Monday morning: add `[[nodiscard]]` to one header and read what the compiler says.

(pause)

DO: Time check: 1:20. If you're past 1:23, compress slides 48 to 52 to their IF BEHIND lines and protect the three spaceship slides.

(beat)

Now the small C++20 and C++23 features. Fifteen minutes, and three of them are the spaceship operator.

42. Designated initializers • C++20 • target 1:20, ~1.5 min

Left side: the C++11 sensor table. `{"rpm", "rpm", 0.0, 12000.0}`, and the comment asks the right question: which double is which? Is 0.0 the minimum or the maximum? Is the second string the units or a display name? You go read the struct.

Right side: `.name = "rpm", .units = "rpm", .min_valid = 0.0, .max_valid = 12'000.0`. **The initializer names the fields, so a table of these documents itself, and swapping two doubles becomes visible in review.**

The rules, because they're stricter than C. Aggregates only. **Designators must be in declaration order.** No out-of-order, no array index designators, no nested `.a.b` designators. That's deliberate: C++ initializes members in declaration order, and the language doesn't want the source to suggest a different order.

DO: Godbolt tab with `designated_init.cpp`. Uncomment line 19, `SensorConfig bad{.units = "V", .name = "x"};`. Expect, on GCC: `error: designator order for field 'SensorConfig::name' does not match declaration order in 'SensorConfig'`. Clang makes it a `-Wreorder-init-list` warning, which our `-Werror` turns into an error. Recovery: read this line aloud.

(beat)

Skipped members get their default member initializer if they have one, and otherwise are value-initialized, which for a double is zero. But under `-Wextra`, both compilers warn about missing fields, so under this repo's flags you name them all, which is what you want in a configuration table anyway.

(pause)

IF AHEAD: GCC and Clang both accept designated initializers in C++17 mode as an extension, with a `-Wpedantic` warning, so under our flags that's an error. Don't rely on the extension in a codebase that has to build with other compilers. And the sensor table is a natural `constexpr` array of these, which Session 3 checks at compile time.

IF BEHIND: "Designated initializers name each field in declaration order; perfect for configuration tables."

Now the three slides that deserve the time in this segment.

43. Three-way comparison, 1: the problem • C++20 • target 1:22, ~2 min

Left side. A `Version` struct with three ints, and six hand-written operators. Equality compares all three members. Less-than compares major, then minor, then patch. Then four more built from those two.

This is exactly what the starter's `Record` has: six operators over four members. That's exercise task 2.

ASK: "Count the ways to get the left side wrong." Give them 15 seconds in chat. Expect: "forget a member in `==`," "compare members in a different order in `<` than in `==`," "copy-paste error in one of the derived ones."

(pause)

All of those. Forget a member in `==`, and two different versions compare equal. Order members differently in `<` than in `==`, and sorting disagrees with equality. Add a fourth member next year and update five of the six operators. And a subtle one: `<=` written as `!(b < a)` is only correct if the type is totally ordered. With a double member and a NaN, `!(b < a)` is true when neither is less, so `<=` says true for values that aren't comparable at all.

Right side. **One line: `auto operator<=>(const Version20&) const = default;`. The compiler generates `<=>` and `==` memberwise, in declaration order. It can't be inconsistent.**

DO: Switch to `spaceship.cpp`, run `./build/demos/s01/demo_s01_spaceship`. Expect `a<b 1 a==b 0 a>=b 0` twice: once for the C++20 struct, once for the C++11 struct. Point out the two lines match.

Same answers. Six functions replaced by one line.

Monday morning, from the segment takeaway: delete six operators from one struct.

IF BEHIND: "Six hand-written comparison operators become one defaulted `operator<=>`, which can't be inconsistent."

44. Three-way comparison, 2: rewriting and `==` • C++20 • target 1:24, ~2 min • SLOW DOWN

This is the slide people get wrong for years, so slow.

The problem line: **you declare one or two operators, and the compiler rewrites the other comparisons in terms of them.** `<=>` returns an ordering, which you compare against zero.

So, with only `<=>` declared: `a < b` becomes `(a <=> b) < 0`. `a >= b` becomes `(a <=> b) >= 0`. The four relational operators all come from `<=>`.

The third comment line, about swapped operands, needs a correction. For two objects of the same type, `b > a` simply becomes `(b <=> a) > 0`. The swap is for mixed types. **If `Version` has `operator<=>(int)`, then `42 > v` has no direct match, so the compiler tries the reversed form, `0 > (v <=> 42)`.** That's how one member operator handles both `v < 42` and `42 < v`, which used to take a pair of friend functions.

(pause)

Now the important part. **The compiler never derives `==` from `<=>`.** They're separate.

Two operators, two jobs. `<=>` gives you the four relational operators. `==` gives you `==` and `!=`. And `!=` is rewritten as `!(a == b)`, so you never write `!=` again either.

Here's the trap. A *defaulted* `<=>` also implicitly declares a defaulted `==`. So with `= default`, you get everything. But **a hand-written `<=>` gives you no `==` at all.** Then `a == b` is a compile error: "no match for operator`==`."

ASK: "Why would the committee make you write `==` separately? Why not derive it?"
Expect someone to say "performance." Say: "Yes. Equality can bail out early. Two strings of different lengths are unequal without looking at a single character. Ordering can't do that; to know which comes first you have to compare characters. Deriving `==` from `<=>` would make every equality check as slow as an ordering."

(pause)

So the rule for your own types: **default both if you can. If you hand-write `<=>`, also write or default `==`.**

IF AHEAD: Rewritten candidates also explain a C++20 migration surprise. Code with an asymmetric `operator==`, for example a member `bool operator==(const Derived&) const` in a class compared against a base, can become ambiguous in C++20 because the reversed candidate now competes. GCC and Clang report it as an ambiguity or a warning (-Wambiguous - reversed-operator on Clang). The fix is to make the parameter types symmetric and the function `const`. If a codebase has hand-written comparison operators that aren't `const`, expect a few of these when the flag moves to 20.

IF BEHIND: "`<=>` gives the four relational operators, `==` gives `==` and `!=`; defaulting `<=>` also defaults `==`, but a hand-written one doesn't."

45. Three-way comparison, 3: categories and member order • C++20 • target 1:26, ~2 min • SLOW DOWN

The return type of `<=>` tells you what kind of ordering you have. Three categories.

strong_ordering: equal means substitutable. Ints. If two ints compare equal, you can swap one for the other anywhere.

weak_ordering: equivalent but distinguishable. A case-insensitive string: "ABC" and "abc" sort the same, but they're not the same string.

partial_ordering: some pairs are unordered. Doubles, because NaN is neither less than, equal to, nor greater than anything, including itself.

Look at the Reading struct. A string and a double. It spells out `partial_ordering`, and the comment says `auto` deduces the same thing. **With `auto`, the defaulted `<=>` returns the weakest category among the members.** One double anywhere in the struct makes the whole thing partially ordered.

Record in the exercise has a double. So its defaulted `<=>` is partial. That's fine for sorting, as long as there are no NaNs in the data. If NaNs can get in, `std::sort` has undefined behavior regardless of whether you used `<=>` or hand-wrote `<`, because NaN breaks strict weak ordering. Validating input before sorting is the real fix.

(pause)

Second block. This is the trap. `struct Bad: a double value, then a long long ts.`

Members compare in declaration order. So this sorts by value first, then timestamp. If you meant "sort by time," you just got "sort by value," and it compiles, and it passes any test that only checks equality.

DO: Run `./build/demos/s01/demo_s01_spaceship_details`. Expect `1 1 1`. Then the godbolt tab with the `Bad` lines added to `main`: run, expect `x<y 0` (`x` has the bigger value). Swap the two member declarations in `Bad` so `ts` comes first. Run: `x<y 1` (`x` has the smaller timestamp). Recovery: "Same source, member order swapped, opposite answer. That's the whole slide."

The member order is the comparison order. When you default `<=>`, the struct layout becomes part of the behavior. In code review, a reordering of members in a struct with a defaulted `<=>` is a behavior change, even if it looks like tidying up for padding.

In the exercise, the starter compares timestamp, then sensor, then value, then status, which is exactly the member order. So defaulting is a pure deletion. Check that before you default.

IF AHEAD: If the order you need isn't the member order, don't reorder the struct. Write `<=>` by hand using a tuple: `return std::tie(ts, value) <=> std::tie(o.ts, o.value);` and then `bool operator==(const Bad&) const = default;`. `std::tie` makes a tuple of references and tuples already have `<=>`. For the NaN problem, C++20 also gives you `std::strong_order` on doubles, which is a total order using the IEEE 754 totalOrder rules, so NaNs sort to a consistent place. Use it as a projection when you need to sort data that might contain NaN.

IF BEHIND: "`auto` gives the weakest category, so a double makes it partial; and members compare in declaration order, so struct layout is now behavior."

Next, a dangling bug that C++20 lets you avoid and C++23 finally removes.

46. Range-for with initializer • C++20 • target 1:28, ~1.5 min

The commented-out line at the top: `for (const auto& n : load().items())`. Looks completely normal.

ASK: "What's wrong with it?" Pause for chat. Expect "load() returns a temporary."

(pause)

load() returns a temporary Batch. .items() returns a reference into it. The temporary dies at the end of the range expression, before the first iteration. You loop over freed memory. The range-for is defined as binding `auto&& __range = load().items();`, and lifetime extension only applies to the object a reference binds to directly, which is the vector reference, not the Batch.

DO: Godbolt range-for tab with ASan. Run. Expect ERROR: AddressSanitizer: stack-use-after-scope. Recovery: "GCC's -Wall even warns about this one, -Wdangling-reference, which is why I took -Werror off this tab."

The C++20 fix is the second block: `for (auto batch = load(); const auto& n : batch.items())`. **Name the temporary in the init statement, and it lives for the whole loop.**

And then C++23, paper P2718, extends the lifetime of every temporary in the range expression to the end of the loop, so the original line becomes correct. But only on GCC 15 and Clang 19 or later. On the GCC 14 and Clang 18 we're using, it still dangles. **Until your compiler has P2718, write the init statement.**

IF AHEAD: If you have a GCC 15 tab handy, switch the compiler on the godbolt tab and the ASan error disappears with no source change. That's a nice way to show that "C++23" on the command line means "the C++23 your compiler implements." And GCC's -Wdangling-reference, in -Wall since GCC 13, catches a useful share of these at compile time, with some false positives on functions that return references to their arguments.

IF BEHIND: "A range-for over a member of a temporary dangles before C++23 compilers; name the temporary in the C++20 init statement."

47. using enum • C++20 • target 1:29, ~1 min

Left: a switch over Status where every case says `Status::Ok`, `Status::Suspect`, `Status::Fault`. Right: `using enum Status;` at the top of the function, and the cases say `Ok`, `Suspect`, `Fault`.

using enum brings the enumerators into the current scope, for that block only. It's the right answer to "enum class is too verbose in switches," which people used to solve by going back to plain enums and losing the type safety.

(pause)

That's exercise task 8, for the switches over Status and ParseError.

(beat)

IF AHEAD: It works at class scope too, which is handy for a class that wraps an enum. And it's scoped like a using-declaration, so two `using enum` statements in the same block with clashing enumerator names is a compile error, which is what you'd want.

IF BEHIND: "using enum Status; drops the prefix inside one function."

48. `[[likely]]`, `[[unlikely]]`, `[[no_unique_address]]` • C++20 • target 1:30, ~1 min

Fast slide.

`[[unlikely]]` after the condition on line 1 tells the compiler this error path is rare, and it lays out the code so the hot path falls straight through. **Honest advice: use them on error paths in hot loops, and measure. Profile-guided optimization beats guesses.** A wrong hint makes code slower, and hints tend to go stale as code changes.

(pause)

`[[no_unique_address]]` lets an empty member, like a stateless deleter or allocator, take zero bytes. So `sizeof(Handle)` equals `sizeof(int)` on GCC and Clang. It's the empty base class trick without inheritance. MSVC ignores the standard spelling for ABI reasons and needs `[[msvc::no_unique_address]]`. Most application code never writes it.

(beat)

IF BEHIND: "Likely and unlikely are layout hints you should measure; `no_unique_address` is for library authors."

49. Know they exist • C++20 • target 1:31, ~1 min

Name-drop slide. Recognize these in review.

char8_t is the one that bites during migration. In C++20, `u8" . . . "` literals are arrays of `char8_t`, not `char`, so code that assigned them to `const char*` stops compiling when you move to 20. Both compilers have `-fno-char8_t` as an escape hatch.

(pause)

`__VA_OPT__` makes variadic macros work cleanly with zero arguments. `constexpr` and `constexpr` are Session 3.

A wording fix on the fourth bullet: in C++20, `[=]` still captures this implicitly. **That implicit capture is deprecated, not removed.** Write `[=, this]` to say it explicitly, or `[=, *this]` to copy the object. Both compilers warn on the implicit form under `-std=c++20`.

And aggregates can now be initialized with parentheses: `Record(1, "rpm", 0.0, Status::Ok)`. That makes `make_unique` and `emplace_back` work with aggregates, which they couldn't before.

IF BEHIND: "char8_t can break u8 literals on migration; the rest are for recognition."

50. The small C++23 features • C++23 • target 1:32, ~1.5 min

Three quality-of-life additions, and one new way to say "this can't happen."

Line 2: `0uz`. **`uz` is a `size_t` literal.** `for (auto i = 0uz; i < v.size(); ++i)`. No more signed/unsigned comparison warning in index loops, and no more `std::size_t i = 0` spelled out.

`auto(x)`: an explicit decay copy, a prvalue copy of `x`. **Where `auto(x)` earns its place is when you pass an element of a container to a function that takes it by reference while modifying that same container.**

Look at the `std::erase(v, auto(v.front()))` line. `std::erase` takes the value by const reference. Without the `auto(...)`, that's a reference into `v`, to the very element the erase is about to move over. Partway through, the element at the front gets overwritten, and now you're erasing a different value. The demo's main has four strings with a duplicate of the front one: with `auto(...)` two remain, without it three do. **Write `auto(v.front())`, and you pass a copy, and it's correct.**

(pause)

`std::to_underlying(Level::High)` replaces `static_cast<std::underlying_type_t<Level>>(...)`.

And `std::unreachable()`. Look at `classify`: three ifs that cover every int, and then `unreachable`. **It's a promise to the optimizer, and if you break the promise it's undefined behavior.** The pattern is to assert the condition first, so debug builds trap, and use `unreachable` for release.

IF AHEAD: Be careful with `unreachable` in safety-related code. Undefined behavior means the optimizer may delete the checks that lead to it. A common defensive alternative in our kind of code is a function that logs and calls `std::abort()` in every build. Use `unreachable` only where you've measured that the branch matters and the invariant is guaranteed elsewhere.

IF BEHIND: "`uz` for index loops, `auto(x)` to copy before you alias, `to_underlying` for enums, `unreachable` as a UB-backed promise."

51. Preprocessor and grammar tidying • C++23 • target 1:33, ~0.5 min

Under a minute. `#elifdef` and `#elifndef`, finally symmetrical with `#ifdef`. `#warning`, which every compiler supported as an extension, is now standard.

Also in C++23: `[[assume(expr)]]`, a promise to the optimizer that the expression is true, undefined behavior if it isn't. It's GCC 13 and up and Clang 19 and up. **On Clang 18 it's an unknown-attribute warning, which under `-Werror` is a build break, so it stays out of the compiled demos.**

(pause)

Labels at the end of a block, and alias declarations in `if` initializers.

(beat)

IF BEHIND: Skip the talk track; say "Preprocessor cleanups, and `[[assume]]` needs Clang 19," and advance.

52. Deprecations worth knowing today • C++20/23 • target 1:34, ~1 min

The volatile story is the one for this room. C++20 deprecated compound assignment and increment on volatile: `v += 1`, `v |= mask`, `v++`. The reasoning: those look like one operation but they're a read and a write, and people assume they're atomic.

The embedded community pushed back hard, because register code is full of `reg |= BIT`. So the bitwise compound operators were un-deprecated by paper P2327, and the rest of the compound assignments by a follow-up core issue, CWG 2654, both applied as defect reports, so current compilers don't warn on `v += 1` even under `-std=c++20`. **But ++ and -- on a volatile are still deprecated.**

(pause)

Both compilers warn: `-Wvolatile` on GCC, `-Wdeprecated-volatile` on Clang. `v = v + 1` is the spelling that says honestly what happens.

`std::aligned_storage` is deprecated in C++23; use `alignas` on a `std::byte` array.

And a correction to the last bullet. `std::result_of` and `std::uncaught_exception` were removed in C++20. `std::iterator`, the base class, was deprecated in C++17 but not removed. It still compiles with a deprecation warning in C++23.

(beat)

IF BEHIND: "volatile increment is still deprecated, `aligned_storage` is deprecated, and the compiler will tell you about both."

53. C++20 and C++23 small features: takeaway • takeaway • target 1:34, ~0.5 min

Use now: `operator<=>` on every value type, designated initializers on configuration tables, using `enum` in switches, `uz` in index loops.

Know for review: `[[likely]]`, `char8_t`, `[=, this]`, the volatile rules.

Monday morning: delete six operators from one struct.

(pause)

And you're going to do exactly that in the next twenty minutes.

54. Exercise: modernize the syntax • exercise • target 1:35, ~20 min

See **Exercise coaching** below for the full run of this segment. The spoken launch:

Open `exercises/s01-modernize-syntax/README.md`. Three tasks in class, in order, and **run the tests after each one**. The command is on the slide.

One thing the slide doesn't say: **the starter builds as C++11**. Task 1 is fine in C++11. Before task 2, open `starter/CMakeLists.txt` and change the standard from 11 to 23, rebuild, and confirm the tests are still green before you touch anything else. The README covers this too.

(pause)

Task 1, `typedef` to `using`. Task 2, the six operators on `Record` become one defaulted operator `<=>`. Task 3, structured bindings and `if` with initializer in `load_stream` and `compute_stats`.

Tasks 4 to 9 are for home. The `solution/` folder is our version, and it's next session's starter.

Go.

DO: Start a visible 20-minute timer. Follow the coaching section.

55. Session 1 takeaway • takeaway • target 1:55, ~5 min (including debrief)

DO: First run the debrief from the coaching section (about 3 minutes), then show this slide.

Here's the whole session in a sentence.

Almost every line of a modern function looks slightly different from its C++11 form, and each difference removes a class of bug. Unused results: `[[nodiscard]]`. Forgotten break: `[[fallthrough]]` plus the warning. Dangling globals in headers: inline variables. Six-way comparison boilerplate: `operator<=>`.

(pause)

Next session: the types that replace raw pointers, sentinel values, out-parameters, and `printf`. `optional`, `variant`, `expected`, `string_view`, `span`, `format`. Your telemetry program's parser is going to start returning `std::expected` instead of a `bool` and an out-parameter.

Bring your solution. If you didn't finish, start from ours; it's in `solution/`.

DO: Point at `handouts/feature-timeline.md`. "Every feature from today is on it, tagged with the session and the standard."

And one Monday-morning thing, pick whichever fits your codebase: `[[nodiscard]]` on one header, `make_unique` through `clang-tidy`, or six operators off one struct.

Thanks, everyone. I'll stay on for a few minutes for questions and for anyone whose build didn't work.

Exercise coaching

Launch in 60 seconds (1:35). Say the launch text on slide 54, including the C++11-to-23 standard switch before task 2. Paste the test command into chat: `cmake --build build && ctest --test-dir build -R s01 --output-on-failure`. Tell people who couldn't build to pair with someone or use Compiler Explorer on `record.h` alone for task 2. Start the timer.

What to say while they work. Keep it quiet for the first five minutes. Then, once, in chat: "Reminder: run the tests after each task, not at the end. If the tests go red you want to know which task did it."

What to watch for (ask people to share a screen or paste errors in chat):

- **Task 1:** people miss the third typedef. The README names `Timestamp` and `StatsBySensor` and says "any others you find." A typedef of a function pointer or an iterator type is the likely third; using `F = void (*)(int);` reads left to right.
- **Before task 2:** forgetting to switch the starter to C++23. Symptom: `<compare>` missing or "'operator<=>' does not name a type" style errors. Fix: the standard in `starter/CMakeLists.txt`, then reconfigure.
- **Task 2:** forgetting `#include <compare>`. With a defaulted `<=>` and an auto return, that's an error, because the category types live there.
- **Task 2:** leaving one old operator declaration in `record.h`, or one definition in `record.cpp`. A leftover free operator `<` competes with the rewritten candidate from `<=>` and either wins silently or produces an ambiguity error. Tell them: delete all six declarations and all six definitions.
- **Task 2:** declaring the defaulted `<=>` outside the struct, or with a non-const signature. It must be `auto operator<=>(const Record&) const = default;` inside the class, or a defaulted friend.
- **Task 2:** someone asks why the result is `partial_ordering`. Answer: value is a double. That's expected and fine. Point back at slide 45.
- **Task 3:** trying to bind into existing variables, like `[it, inserted] = counts.insert(...)` without `auto`. Structured bindings always declare new names; if they need existing variables, that's `std::tie`.
- **Task 3:** putting the binding before the `if` in `load_stream` instead of in the `if` initializer. It works, but the README asks for the scoped version, and the point is scoping.
- **Task 3 bonus:** `try_emplace` and output order. It doesn't change the map's order, so the report should be identical.

Five-minutes-left call (1:50). In chat and aloud: "Five minutes. If you're mid-task, get back to green. A passing build with two tasks done beats a broken build with three."

Debrief with the solution (1:55, about 3 minutes).

DO: Open `exercises/s01-modernize-syntax/solution/record.h`. Point at the single defaulted `operator<=>` line where six declarations used to be. Then show `record.cpp` and note the operator definitions are simply gone.

Say: **"The whole of task 2 is one line added and about twenty lines deleted, and the comparison test is the proof nothing changed."**

DO: If there's time, run the starter and solution binaries on `data/sample.csv` and `diff` the outputs. The README says they're byte-identical. Expect no diff output.

Recovery: if the solution binary name or data path isn't what you expect, skip it and say "the README's diff check is part of tonight's homework."

ASK: "Who got all three green?" Then: "Who hit the leftover-operator ambiguity?"

Expect at least one. Say: "That's the most common one. And notice the compiler caught it, not the tests."

Then point at tasks 4 to 9 for home. Tasks 4 (`[[nodiscard]]`) and 5 (`from_chars`) are the ones most likely to find something real. For task 5, remind them of slide 38: `from_chars` rejects a leading `+` and writes the output on trailing garbage, so if the tests go red, that's why.

Closing. Go to slide 55.

Likely questions and answers

Q: Our certified toolchain is an older GCC. What from today can we actually use? Use the badges and the cppreference compiler support table. As a rough guide, GCC 5 has all of C++14, and GCC 7 has nearly all of the C++17 language features from today; GCC 8 adds integer `from_chars`, and floating-point `from_chars` needs GCC 11's `libstdc++`. Most of today's C++20 language features (`<=>`, designated initializers, `using enum`, range-for init) need GCC 10 or 11. Check your exact version against the table before you put a feature in a coding standard.

Q: If we move from `-std=c++14` to `-std=c++17` or 20, do we break ABI with libraries built the old way? Mostly no for `libstdc++` on GCC, which aims to keep the ABI stable across `-std` levels for features it considers stable, but C++17 support only became non-experimental in GCC 9 and C++20 later, so mixing objects from older experimental modes is risky. One concrete break: `noexcept` became part of the function type in C++17, so function pointer types mangle differently, and GCC warns with `-Wnoexcept-type`. The safe policy is to build a whole program, including its static libraries, at one standard level.

Q: Is a defaulted operator `<=>` slower than hand-written operators? No, in practice. `a < b` becomes a memberwise lexicographic comparison that stops at the first difference, the same as a hand-written `operator<`, and the defaulted `==` is a separate memberwise equality that also stops early. For strings, `<=>` does one compare instead of the two `<` calls a naive hand-written version might make. If a hot path matters, look at the generated code on Compiler Explorer; it's usually identical.

Q: How does this line up with MISRA or AUTOSAR? AUTOSAR C++14 targets C++14, and MISRA C++:2023 targets C++17, so structured bindings, `if` with initializer, `[[nodiscard]]`, and `inline` variables are all within its language baseline. Several of today's features enforce rules those standards already have, such as checking return values and avoiding implicit fall-through. C++20 and 23 features are outside both baselines, so a project bound to one of them

needs a deviation or has to wait. Check the specific rules your program is held to rather than assuming.

Q: Does `[[nodiscard]]` or `[[likely]]` cost anything at runtime? `[[nodiscard]]`, `[[maybe_unused]]`, `[[fallthrough]]`, and `[[deprecated]]` are purely compile-time diagnostics and generate no code. `[[likely]]` and `[[unlikely]]` can change code layout and branch ordering, which can make code faster or slower, so measure. `[[no_unique_address]]` changes object layout, which is an ABI change for that type.

Q: Isn't "almost always auto" bad for readability? It depends where. Use `auto` where the type is already on the line (`make_unique`, casts, iterators) or unspellable (lambdas), and spell the type where it's information the reader needs, especially in interfaces. The slide 8 rule matters more than the style debate: `const auto&` when reading, `auto&` when mutating, plain `auto` only for a deliberate copy.

Q: Can we use designated initializers or using `enum` in C++17 mode as extensions? GCC and Clang accept designated initializers in C++17 mode with a pedantic warning, which is an error under `-Wpedantic -Werror`. `using enum` isn't available before C++20. Relying on extensions ties you to those compilers and to non-pedantic flags, which is usually the wrong trade for code that has to live for decades.

Q: Do structured bindings or `if-with-initializer` add any overhead? No. A structured binding is one hidden object plus names that alias into it, and the optimizer sees through it completely. `if` with initializer is purely about scope. Both are free; check it on Compiler Explorer if someone doubts it.

Q: Why does the exercise starter build as C++11 instead of C++23? So that the starting point is honest C++11 that a C++11 compiler accepts, and so the first step of modernization, changing the standard flag and confirming the tests stay green, is part of the exercise. That's also the right order in a real codebase: move the flag, prove nothing changed, then start using features.

Q: Is `std::from_chars` always faster than `strtod`? Usually, and often by a lot for floating point, because it skips locale handling and `errno`, and recent `libstdc++` uses a fast parsing algorithm. It's also stricter: no leading whitespace, no leading `+`, and no `0x` prefix, even in hex mode. Treat the switch as a behavior change and let the tests decide.

Q: How do we roll this out across a large codebase without a big-bang rewrite? Move the standard flag first, alone, and fix only what breaks. Then turn on targeted clang-tidy `modernize-*` checks one at a time, starting with low-risk ones like `modernize-make-unique`, `modernize-use-override`, and `modernize-concat-nested-namespaces`. Add `[[nodiscard]]` header by header. Session 5 covers a full adoption roadmap, and there's a worksheet in the handouts.

Deck issues found

Status: fixed in the deck, demos, outline and README on 2026-10-02 (see scripts/README.md). Items kept for the record. Still open: the Compiler Explorer <add short link> placeholders in every demo file header.

- **Slide 2 (notes):** says to build the starter "during the first break," but the session has no scheduled break (10+20+25+25+15+20 minutes plus a 5-minute wrap fills 2:00). The script asks people to build during the opening instead.
- **Slide 4 (notes):** "every line of code in this deck is pulled from a compiled file by a script, so nothing on a slide is pseudo-code." False: slides 13, 14, 17 (left), 19, 20, 21, 22, 23, 29, 30, 31, 33, 39, 40, and 48 are hand-typed, and slides 19, 29, 30, and 33 contain . . . Also the bullet lists -Wall -Wextra -Werror while slide 5 adds -Wpedantic.
- **Slide 5:** promises "Compiler Explorer links on every demo slide," but no slide has one and every demos/s01/*.cpp header still reads Compiler Explorer: <add short link>.
- **Slide 6:** run command build/exercises/s01-modernize-syntax/s01_starter data/sample.csv is written from the repo root, while the exercise README runs from the exercise folder with ../../build/..., implying data/ lives under exercises/s01-modernize-syntax/. One of the two paths is probably wrong; verify. (The staged copy has no starter/, solution/, tests/, or data/, so this, the "seven rejections" claim, and the task details could not be checked.)
- **Slide 8:** "auto x{42}; // int since C++17 (initializer_list<int> in 11/14)": the change (N3922) was a defect report, and GCC and Clang give int even under -std=c++11 (verified). Accurate per the original C++11/14 text only.
- **Slide 11:** "Two atomic increments per copy" is wrong: a shared_ptr copy does one atomic increment; the matching decrement happens at destruction.
- **Slide 17:** the before and after don't do the same thing (left sorts Records by value descending, right sorts anything by .size() ascending), contradicting slide 4's "doing the same thing."
- **Slide 23:** "Works on functions, types, variables, enumerators, namespaces": enumerators and namespaces were added in C++17, not C++14.
- **Slide 26 (notes):** "the C++17 segment has two slides that can be skipped (evaluation order, removals)": there's no separate removals slide; removals are folded into the slide 41 takeaway.
- **Slide 35 (notes):** "-Wimplicit-fallthrough is in -Wextra on both compilers": true for GCC only. Clang 18's -Wextra doesn't enable it (verified: GCC warns, Clang is silent).
- **Slide 38 (notes):** the locale example is backwards. In a German locale, strtod("41,25") gives 41.25 and strtod("41.25") gives 41. "Allocation" also doesn't disappear in this specific before/after, since the C++11 version takes a const std::string& and doesn't allocate.
- **Slide 38 / demos/s01/from_chars.cpp:** main reads a and b as printf arguments in the same call that parses into them; argument evaluation order is unspecified, so GCC prints 1 0 0.00 0.00 and Clang prints 1 0 41.25 41.25 (verified). Also, the "after" version writes *out on trailing garbage ("41.25x" leaves 41.25 in b) where the

"before" doesn't, so the two aren't behavior-identical; and `from_chars` rejects a leading + that `strtod` accepts.

- **Slide 39:** `buffer.substr_view(0, n)` doesn't exist on `std::string` (verified: compile error). Use `std::string_view(buffer).substr(0, n)`.
- **Slide 40:** "`s = s + f(s)` behaves" is doubtful: C++17 sequences the right side of `=` before the left, but the operands of `+` are still unordered, so if `f` modifies `s` the result is still unspecified. Better example: `m[k] = m.size()`; . The notes also credit the leak fix to "callee-before-arguments"; the actual fix is that function arguments are now indeterminately sequenced (no interleaving).
- **Slide 41 (notes):** "The next segment is 13 slides in 15 minutes": it's 12 (slides 42 to 53).
- **Slide 44 / demos/s01/spaceship_details.cpp:** "`b > a` becomes `(a <=> b) < 0` (operands may be swapped)" is misleading. For same-type operands the non-reversed `(b <=> a) > 0` is chosen; the reversed form matters for mixed types such as `42 > v` becoming `0 > (v <=> 42)`.
- **Slide 49:** "`[=]` no longer implicitly captures this (deprecated)" contradicts itself: in C++20 `[=]` still captures this implicitly; that capture is deprecated.
- **Slide 50:** the problem line says "Four quality-of-life additions" but the slide shows three (`uz`, `auto(x)`, `to_underlying`) plus unreachable. And the `auto(x)` example doesn't motivate the feature: `auto copy = v.front()`; already copies. The real case is passing a self-referencing element to a by-reference parameter, e.g. `std::remove(v.begin(), v.end(), auto(v.front()))` (verified: without `auto(...)` the result is 2 1 3 1 4, with it 2 3 4).
- **Slide 52:** "C++17 deprecated, C++20 removed: `std::iterator`": `std::iterator` was deprecated in C++17 but not removed; it still compiles with a deprecation warning under `-std=c++23` on both compilers (verified). Also the notes credit P2327 alone; P2327 un-deprecated only the bitwise compound operators, and CWG 2654 (also a DR) un-deprecated the rest.
- **Slide 54:** omits that the starter builds as C++11 and must be switched to C++23 before task 2 (`<compare>` and `operator<=>`), which the README states. This will be the most common in-class blocker.